Dynamic Modeling for Data-Driven Test Prioritization in Automotive Validation

Scientific thesis for the procurance of the degree M.Sc.
from the TUM School of Computation, Information and Technology
at the Technical University of Munich.

Supervised by Univ.-Prof. Dr.-Ing. Sandra Hirche
M.Sc. Max Beier
Chair of Information-Oriented Control

M.Sc. Nils Mayat
Dr.-Ing. Tim Weisser
BMW Group

Submitted by B.Sc. Aleksandar Marjanovic
aleksandar.marjanovic@tum.de

Submitted on Munich, 02.03.2026

TECHNISCHE UNIVERSITÄT MÜNCHEN
LEHRSTUHL FÜR INFORMATIONSTECHNISCHE REGELUNG
ORDINARIA: UNIV.-PROF. DR.-ING. SANDRA HIRCHE

February 22, 2026

MASTER'S THESIS

Dynamic Modeling for Data-Driven Test Prioritization in Automotive Validation

Problem description:

The increase in product complexity and ever-shorter development times in the automotive industry are creating a conflict in the quality-cost-time triangle, with quality suffering as a result. The significant increase in complexity is particularly evident in the E/E sector [1], where there has been exponential growth in functions and system networking. Modern vehicles, even in the low-end segment, now have up to 100 electronic control units (ECUs) and 100 million lines of code, as features such as adaptive cruise control and automatic emergency braking become standard [2]. In order to ensure quality for the customer on time, the validation process for the entire system must be designed to be as effective and efficient as possible. As part of this master's thesis, a data-driven approach to modeling and controlling the validation process is to be developed in the context of design science research methodology [3]. Machine learning methods will be used to map the validation process as a dynamic system, identify critical influencing factors, and enable predictions about test prioritization and error slips. The goal is to develop, interpret, and systematically evaluate a suitable model for data-based decision support. The developed modeling methodology will be validated using historical test data (e.g., error rates, verification times) from a German OEM and compared in terms of its practicality and performance.

Tasks:

- Systematic literature review on current methods
- Processing and analysis of historical test and defect data
- Development of a mathematical model for the available data
- Selection and development of suitable machine learning methods to train the model
- Implementation and training of chosen ML approaches
- Systematic evaluation and comparison to baselines using relevant performance metrics
- Recommendations for process improvement

Bibliography:

[1] Askaripoor et al.
E/E Architecture Synthesis: Challenges and Technologies.
Electronics, 11(4), 2022.

[2] Charette.
How Software Is Eating the Car.
IEEE Spectrum, 2021.

[3] Peffers et al.
A Design Science Research Methodology for Information Systems Research.
Journal of Management Information Systems, 24(3), 2007.

Supervisor:	M. Sc. Max Beier (TUM), M. Sc. Nils Mayat (BMW)
Start:	01.09.2025
Intermediate Report:	28.12.2025
Delivery:	02.03.2026

(S. Hirche)
Univ.-Professor

Abstract

Modern vehicles rely on increasingly complex software systems, making exhaustive validation infeasible under limited time and resource constraints. At the same time, large volumes of structured validation data are generated during development but are rarely used systematically for decision support. This thesis addresses the problem of risk-based test prioritization in automotive software validation. It assumes that historical ticket data contain statistically learnable patterns and that risk can be decomposed into the probability of a defect and its conditional severity. Based on this assumption, the validation process is modeled as a probabilistic system in which ticket occurrence and severity are estimated separately and combined into an expected risk score used for ranking test configurations. To assess the effectiveness of this approach, classical ensemble methods and transformer-based tabular models are evaluated on industrial data. The results show that tree-based methods perform strongly for defect occurrence prediction, while the transformer-based approach improves severity estimation and early detection of critical cases. A hybrid combination achieves the best overall prioritization performance.

Disclaimer on the Use of Artificial Intelligence: ChatGPT and Grammarly were used for grammar correction and proofreading. No AI-generated content was used in producing this report.

Contents

1 Introduction **7**
1.1 Problem Statement 8
1.2 Structure of the Thesis 10

2 Literature Review and Theoretical Background **11**
2.1 Automotive Validation and Test Case Prioritization 11
2.2 Machine Learning for Tabular Data 13
2.3 Research gap 15

3 Technical Approach **17**
3.1 Process and Design 17
3.2 Data Preprocessing and Dataset Construction 20
3.3 Mathematical Modeling 23
3.3.1 Probabilistic Model for Ticket Occurrence, Severity, and Risk 25
3.3.2 Prior Classifier Baseline (Empirical Prior Model) 28
3.3.3 Random Forest Baseline (Nonlinear Feature-Based Ensemble) 29
3.3.4 Prior-Data Fitted Networks (PFN) for Hierarchical Risk Inference 30
3.3.5 Hybrid Risk Model via Model Specialization 32
3.4 Implementation 33
3.5 Evaluation Metrics 36

4 Setup and Results **39**
4.1 Experimental Setup 39
4.2 Ticket Occurrence Modeling 43
4.3 Severity Modeling 45
4.4 Risk-Based Prioritization 47

5 Discussion **53**
5.1 Interpretation of Key Findings 53
5.2 Implications and Comparison with Related Work 55

6 Conclusion **57**

A Validation Workflow **59**

List of Figures **61**

Bibliography **63**

Chapter 1

Introduction

> "We suffer more often in imagination than in reality."
>
> — Seneca [1]

In recent years, the pace of technological innovation has increased substantially. New technologies are being integrated into everyday life at an unprecedented rate, requiring organizations to respond with greater flexibility and efficiency. Companies can no longer rely on rigid structures or slow decision-making processes, instead, they must continuously adapt to remain competitive in an environment shaped by ongoing technological development and innovation [2].
Modern vehicles are undergoing a fundamental transition from predominantly hardware-centric systems toward software-centric platforms. This shift is driven by electrification, connectivity, advanced driver assistance systems (ADAS), and the emerging paradigm of software-defined vehicles (SDVs). In SDVs, a substantial share of vehicle functionality is implemented, configured, and updated through software, leading to a significant increase in software complexity and functionality within electrical/electronic (E/E) architectures [2].
As a consequence, software in the automotive industry is no longer a subordinate component but a strategic asset that decisively differentiates products. This transformation results in a growing number of software components, including electronic control units (ECUs), as well as increasingly complex dependencies and interactions that must be correctly implemented and rigorously verified to ensure vehicle safety and reliability. Modern vehicles may integrate tens or even hundreds of ECUs and comprise up to hundreds of millions of lines of software code, which further amplifies the challenge of ensuring correct behavior under all operating conditions [2].
This software-centric development has a profound impact on verification and validation (V&V) activities. Functional safety standards such as ISO 26262 mandate systematic verification and validation throughout the entire development lifecycle in order to minimize hazards and residual risks. However, increasing software complexity renders comprehensive verification more expensive, labor-intensive, and time-consuming. In particular, late integration testing and manual analysis procedures

often lead to delayed feedback, limited fault localization, and rising development costs in later phases [3].
At the same time, competitive pressure within the automotive industry is intensifying, resulting in shorter development cycles and growing market expectations. Original equipment manufacturers (OEMs) and suppliers are required to deliver new vehicle models and frequent software updates within shrinking time frames, while maintaining high levels of quality, functional safety, and regulatory compliance. The combination of increasing system complexity and decreasing time budgets highlights the limitations of traditional quality assurance approaches for software-intensive automotive systems [2].
Concurrently, large volumes of data are continuously generated throughout the development process, including data from version control systems, test executions, fault reports, and integration logs. These data sources contain valuable information that can be exploited for learning-based test prioritization and quality assurance. In the broader software engineering domain, data-driven and machine learning-based methods for software defect prediction and risk prioritization have demonstrated promising results by focusing testing efforts on the most critical components and enabling more cost-effective quality improvements [4].
Taken together, these industrial and technological trends motivate the development of data-driven quality assurance techniques that are specifically tailored to the constraints, complexity, and safety requirements of modern automotive software systems [3].

1.1 Problem Statement

The software-defined transformation, such as the growing number of software functions and the centralization of E/E architectures, is leading to an exponential increase in interactions, which makes traditional verification approaches less scalable. High dependencies between components further increase the likelihood of unexpected errors during system integration or operation in the field [3, 5].
The following challenges deserve attention. First, testing resources are limited. Given increasingly shorter vehicle development cycles and a growing number of variants, comprehensive testing of all software configurations and E/E interactions is no longer feasible. Empirical studies and industry observations indicate that time constraints and insufficient automation hinder exhaustive testing and increase the risk of defects propagating into later development phases or, in the worst case, being delivered to the customer [3, 5].
At the same time, the scale and complexity of validation data have increased substantially. Industrial validation databases contain several hundred thousand historical records, each described by dozens of heterogeneous features. The number of distinct software functions alone is on the order of 10^3, resulting in a highly cross-classified and high-dimensional data space. In such a setting, purely heuristic

prioritization based on empirical knowledge or expert intuition becomes increasingly difficult to maintain in a consistent and scalable manner. While expert judgment remains valuable, it is no longer sufficient as the sole decision mechanism for allocating limited testing resources across a large and evolving function landscape.

Second, data utilization remains insufficient. Although large volumes of development and test data are generated throughout the automotive software lifecycle, this data is rarely exploited systematically for early error detection, risk assessment, or test prioritization. In general software engineering, data-driven techniques, particularly machine learning methods, have been proposed and successfully applied for software defect prediction in order to reduce verification effort and improve the identification of error-prone components [4]. However, their adoption in the automotive domain remains limited due to domain-specific constraints, complex system dependencies, and strict functional safety requirements. In addition, automotive validation environments often suffer from poorly documented defects and incomplete specifications, which further complicates the effective application of learning-based approaches while simultaneously highlighting their potential relevance [3].

The core problem addressed in this work is therefore the absence of a systematic, data-driven strategy for prioritizing testable software functions according to their estimated risk contribution. Given the scale of modern validation datasets and the large number of interacting software functions, it is not feasible to test all configurations with equal intensity. Instead, limited testing capacity must be allocated in a way that minimizes the expected realized risk across the validation process. This requires a probabilistic model that estimates the risk of each candidate function or configuration based on historical data and ranks them accordingly.

The objective of this thesis is to develop such a data-driven prioritization method that sorts the list of candidate test functions by their computed expected risk, thereby concentrating testing effort on the configurations with the highest estimated impact. By aligning test execution order with statistically estimated risk, the approach aims to maximize early coverage of high-severity failures and to reduce the accumulation of critical defects under strict time and resource constraints [4].

Accordingly, this work addresses the following research questions:

1. How can the automotive validation process be represented as a dynamic and probabilistic system suitable for risk-based prioritization?

2. Which machine learning methods are capable of capturing the temporal, hierarchical, and cross-functional structure of large-scale automotive validation data?

3. How effectively can models learned from historical test data estimate expected risk and improve the prioritization of test functions under realistic resource constraints?

1.2 Structure of the Thesis

This thesis is structured into six chapters, moving from theoretical foundations to empirical validation and practical implications.

Chapter 2 reviews related work in automotive validation, test case prioritization, and machine learning for tabular data. It discusses classical and foundation-model approaches and clarifies the research gap addressed in this thesis.

Chapter 3 presents the technical approach. It formalizes the validation process as a probabilistic system, describes data preprocessing and dataset construction, and introduces the modeling framework for ticket occurrence, severity, and expected risk.

Chapter 4 reports the empirical results. The models are evaluated separately for occurrence and severity prediction and subsequently assessed in a risk-based prioritization setting using ranking metrics.

Chapter 5 interprets the results and discusses their implications for industrial validation, model selection, and hybrid strategies.

Chapter 6 summarizes the main contributions, outlines methodological and data-related limitations, and suggests directions for future research.

Chapter 2

Literature Review and Theoretical Background

"There is nothing permanent except change."

— Heraclitus [6]

2.1 Automotive Validation and Test Case Prioritization

Over the past decade, the increasing adoption of data-driven methods has fundamentally changed how software-intensive systems are developed, tested, and validated. This trend is particularly pronounced in safety-critical domains such as automotive software, where growing system complexity, frequent updates, and continuous integration pipelines have amplified the need for scalable and risk-aware validation strategies. Consequently, a broad body of related work has emerged at the intersection of software testing, machine learning, and data-driven quality assurance, spanning regression testing, continuous integration (CI), failure analysis, anomaly detection, and time-series modeling for maintenance and reliability forecasting [3, 4, 7–9].

A large share of prior research originates from the general software engineering community and addresses regression testing and test case prioritization (TCP) as optimization problems. Early foundational work formalized TCP as the task of ordering test cases to maximize early fault detection under limited resources [10, 11]. Traditional approaches rely on code coverage, execution history, or fault frequency and typically evaluate performance using value-neutral metrics such as the Average Percentage of Fault Detection (APFD). While effective in generic software contexts, these assumptions rarely hold in industrial settings, where test cases differ substantially in execution cost and faults vary in severity and business impact.

Building on these foundations, a rich line of research has explored search-based and metaheuristic approaches to TCP, including genetic algorithms, ant colony opti-

mization, particle swarm optimization, and hybrid heuristics [9, 12]. These methods demonstrate consistent improvements over random or greedy baselines across diverse datasets, but they largely remain agnostic to domain-specific notions of risk, safety relevance, and business value. Moreover, most empirical evaluations rely on open-source or academic benchmarks, limiting their direct transferability to highly regulated domains such as automotive software.

To address these limitations, value-based and risk-aware extensions of TCP have been proposed. In these approaches, prioritization explicitly accounts for fault severity, test execution cost, or requirement criticality [11, 13]. Value-based TCP is motivated by the observation that a small fraction of system functionality often accounts for a disproportionately large share of business or safety risk. While these methods provide more realistic prioritization objectives, they are still predominantly evaluated in generic enterprise or web-based software systems and rarely reflect the structure of automotive validation environments.

Automotive-specific research has increasingly recognized these gaps. Some studies investigate regression testing and validation in the presence of complex configurations, non-code artifacts, and system-level interactions typical of automotive E/E architectures [5, 14]. More recent work emphasizes the integration of testing into CI pipelines and data-driven validation workflows, highlighting challenges related to scalability, traceability, and real-time decision-making in industrial validation processes [4, 15]. These studies underline that automotive testing is not only a technical optimization problem but also an organizational and data management challenge in highly configurable and safety-critical environments.

A particularly relevant contribution for this thesis is the work on severity-aware prioritization of system-level regression tests in automotive software [14]. This study explicitly targets automotive systems and proposes a prioritization strategy that incorporates fault severity information to improve early detection of critical failures. Using industrial datasets from automotive software projects, the authors demonstrate that severity-aware prioritization outperforms traditional value-neutral baselines in terms of safety-relevant fault detection. This work closely aligns with the problem setting of this thesis, as it highlights both the availability and the importance of severity annotations in automotive defect data while also exposing limitations related to static prioritization strategies and model adaptability.

Beyond regression testing, related work has explored data-driven failure analysis and anomaly detection, often using time-series data from system logs, sensor streams, or event traces. Techniques such as sparse autoencoders, representation learning for event time series, and unsupervised anomaly detection have been successfully applied to identify abnormal system behavior without explicit fault labels [8]. While these methods are typically positioned outside classical testing workflows, they provide valuable insights into how temporal patterns and historical data can be leveraged for early risk identification. However, they often lack direct mechanisms for actionable test prioritization within validation pipelines.

Machine learning approaches for software defect prediction form another closely re-

lated research stream. According to a comprehensive review [4], supervised learning models, including tree-based methods, neural networks, and ensemble techniques, can effectively predict fault-prone components based on historical metrics. In automotive contexts, these methods are increasingly discussed as enablers for risk-based validation, yet their integration with concrete testing decisions, such as which system-level tests to execute next, remains an open challenge.

Overall, the reviewed literature demonstrates substantial progress in data-driven testing and validation, but also reveals a clear gap at the intersection of automotive software, severity-aware risk modeling, and modern machine learning for tabular industrial data. Existing automotive-focused approaches tend to rely on handcrafted prioritization heuristics or static severity rules, while generic ML-based methods often ignore domain-specific constraints and safety requirements. This thesis builds on these insights by positioning data-driven test prioritization as a dynamic system modeling problem, leveraging historical defect and validation data to support risk-based decision-making in automotive software validation.

2.2 Machine Learning for Tabular Data

Before introducing specific ensemble variants, it is useful to recall the theoretical foundation of decision tree learning. A decision tree partitions the feature space recursively by selecting splits that maximize impurity reduction, typically measured via entropy or the Gini index [16, 17]. The resulting model represents a piecewise-constant approximation of the conditional class distribution and can capture nonlinear feature interactions without requiring explicit feature transformations. However, single decision trees tend to exhibit high variance and sensitivity to small perturbations in the data. Random Forests address this limitation through bootstrap aggregation and randomized feature selection, thereby reducing variance while preserving the expressive partitioning capability of individual trees [18].

Tabular data is the dominant format in many industrial workflows because operational knowledge is naturally stored as structured records such as tickets, configuration metadata, test logs, and measurement tables [19]. Although deep learning has achieved major progress on unstructured modalities such as images and text, strong performance on tabular prediction tasks has long been associated with tree-based approaches, which are widely treated as robust default baselines for heterogeneous feature types [16, 19].

Among these methods, Random Forests [18] represent one of the most established ensemble techniques for tabular classification. A Random Forest consists of multiple decision trees trained on bootstrap samples of the data, where each split considers only a random subset of features. This randomized ensemble construction reduces variance and improves generalization compared to single decision trees. Random Forests are known for their robustness to noise, mixed feature types, and nonlinear interactions, and they have been widely applied in software defect prediction and

industrial classification tasks [4, 16]. Due to their stability and relatively low tuning effort, they are often used as strong classical baselines in empirical comparisons.

In addition to feature-based models, simple label-based baselines are commonly used for reference. A prior or majority classifier predicts outcomes solely based on empirical class frequencies in the training data, without considering input features. In binary classification, this corresponds to predicting the empirical Bernoulli parameter, while in multi-class settings it corresponds to predicting the empirical categorical distribution [16]. Although such models ignore feature information, they provide an important lower bound under proper scoring rules and allow researchers to quantify how much predictive performance is gained from modeling feature-label relationships.

While tree-based ensembles have dominated tabular learning for many years, recent research argues that the field is shifting toward *tabular foundation models* that are trained once to learn transferable priors across many datasets and then applied with minimal per-task adaptation [19–21].

In contrast to classical feature-based learners, Prior-Data Fitted Networks (PFNs) are grounded in Bayesian decision theory and meta-learning. The central idea is to approximate the Bayesian posterior predictive distribution of a learning problem by training a neural network on a distribution of synthetic tasks sampled from a predefined prior [22, 23]. Instead of optimizing model parameters separately for each new dataset, the network learns to perform amortized inference: given a set of labeled examples, it directly predicts the posterior predictive distribution for unseen instances via a single forward pass [24]. This framework connects Transformer-based in-context learning with Bayesian inference, where conditioning on the training set implicitly replaces explicit gradient-based optimization.

A key development enabling this shift is the *Prior-Data Fitted Network* (PFN) framework, in which a Transformer-like model is meta-trained on a distribution of synthetic problems so that inference on a new dataset can be performed directly from a set of labeled examples [24, 25]. TabPFN operationalizes this idea for tabular classification by performing *in-context learning* (ICL). The downstream training set is embedded into the model input, and predictions for new instances are produced without gradient-based optimization at inference time [24].

Because PFN-style models are trained to approximate Bayesian learning procedures under a task prior, they naturally provide probabilistic predictions and can be interpreted as amortized Bayesian inference [20, 25]. This probabilistic formulation supports uncertainty-aware decision making, especially in settings where labeled data may be limited [20, 25].

Empirical results report that TabPFN achieves strong performance on small to medium-sized tabular datasets while requiring substantially less task-specific tuning compared to classical methods [20, 24]. For heterogeneous industrial tables, its relevance lies not only in predictive accuracy but also in its ability to handle mixed feature semantics, varying scales, and changing column identities across datasets [26]. Analyses of TabPFN v2 highlight architectural design choices such as random-

ized feature tokens that help unify heterogeneous schemas into consistent internal representations and promote column-aware behavior [26]. The attention mechanism further enables flexible conditioning on relevant training examples and feature interactions when dependencies are complex [24, 26]. TabPFN has also been extended to time series forecasting by reformulating forecasting problems as tabular regression tasks with lightweight temporal feature engineering [27]. These results indicate that strong forecasting performance can emerge without dedicated time series specific pretraining, relying instead on the model's learned attention-based conditioning [25, 27].

Within the broader landscape of tabular machine learning, recent surveys position foundation models as a complementary layer on top of classical baselines and specialized deep tabular architectures, emphasizing that current progress is increasingly driven by transferability, robustness, and reduced hyperparameter tuning rather than single-dataset optimization [19]. In this context, TabPFN-2.5 is presented as a next-generation tabular foundation model that scales to larger table sizes and reports competitive benchmark performance relative to tuned tree-based ensembles and automated model selection pipelines [21].

Finally, an important methodological dimension concerns the trade-off between pure in-context learning (without parameter updates) and fine-tuning foundation models on target-domain data [28]. Empirical studies suggest that fine-tuning can improve performance in certain settings and alter internal similarity structures, although gains may depend on dataset characteristics and domain shift conditions [28].

2.3 Research gap

Despite the advancements in both automated testing and machine learning, a significant gap remains at the intersection of dynamic automotive validation and state-of-the-art tabular modeling. This research gap can be categorized into three primary dimensions.

First dimension is the Adaptation Gap in Highly Specialized Domains. Existing literature on Test Case Prioritization (TCP) has evolved from basic coverage-based ordering [10] to sophisticated metaheuristic and value-based approaches [9, 13]. However, as noted by [14], these methods often remain agnostic to the specific constraints of automotive E/E architectures, such as complex hardware-software dependencies and varying fault severities. While recent studies have begun to integrate severity-aware prioritization in automotive CI pipelines [3, 14], they predominantly rely on static rules or traditional supervised learning that require extensive historical data and manual feature engineering to remain effective. There is a lack of research on models that can autonomously adapt to the rapidly shifting data distributions inherent in modern vehicle validation.

Second dimension is the Methodological Gap in Tabular Learning. In the broader machine learning community, the focus has shifted toward Tabular Foundation Models like TabPFN, which leverage In-Context Learning (ICL) to perform Bayesian inference without task-specific training [24, 25]. While these models have demonstrated SOTA performance on general benchmarks by learning transferable priors [20] their application to industrial validation metadata remains largely unexplored. Most automotive-specific research still utilizes tree-based baselines like XGBoost or LightGBM, which, while robust, lack the probabilistic flexibility and tuning-free deployment advantages of Prior-Data Fitted Networks [19, 21]. There is currently , based on our knowladge, no established framework that evaluates how these transformer-based priors behave when confronted with the noisy, high-dimensional, and often imbalanced "ticket-to-test" data found in automotive systems.

Third dimension is the Transparency and Process Gap. A critical shortcoming in the current state of research is the lack of transparency regarding industrial datasets. As highlighted by [15], automotive testing is an organizational data management challenge where the exact process of transforming raw logs into actionable prioritization features is rarely documented. Most existing papers utilize proprietary or "sanitized" datasets that obscure the dynamic nature of validation. Consequently, there is a missing link in the literature: a study that not only applies a SOTA foundation model like TabPFN-2.5 to a real-world automotive use case but also rigorously defines the dynamic modeling process, bridging the gap between raw industrial data and automated, risk-based decision-making.

Chapter 3

Technical Approach

"There is no greatness where there is not simplicity, goodness and truth."

— Leo Tolstoy [29]

3.1 Process and Design

To enable meaningful data-driven modeling, it is necessary to abstract the industrial validation process that generates the observed ticket data. Modeling choices should reflect the objectives, constraints, and feedback mechanisms inherent to the automotive software development and validation workflow. This section therefore introduces a simplified yet structurally faithful representation of the process, focusing on how tickets, severities, and temporal information arise and how they relate to decision-making under resource constraints.

The workflow starts with *development requirements*, which specify the intended functional behavior, operational conditions, and quality constraints of electronic control units (ECUs) and their software functions. These requirements form a shared reference for both the software development team and the testing team and define the target system behavior throughout the development lifecycle. Based on the requirements, the testing team derives test specifications and test plans. Due to limited testing capacity, testing is inherently selective, and test planning is guided by the objective of exposing critical failures as early as possible. Early detection is of particular importance, as unresolved issues tend to propagate into later integration stages, where remediation becomes significantly more costly [30].

In parallel, the software development team incrementally develops and integrates software artifacts into the vehicle platform. When failures are detected through testing or integration, remediation efforts are prioritized under constraints on time, personnel, and computational resources. As a result, development typically focuses on resolving the most critical problems first, while lower-severity issues may be deferred to later iterations. The interaction between testing and development is mediated by the integration and deployment process, represented here by the vehicle

production process. Software releases are deployed and tested, producing observable outcomes in the form of successful tests or detected failures with associated criticality levels. These outcomes feed back into both testing and development activities, thereby closing the process loop. Figure 3.1 illustrates this high-level process abstraction.

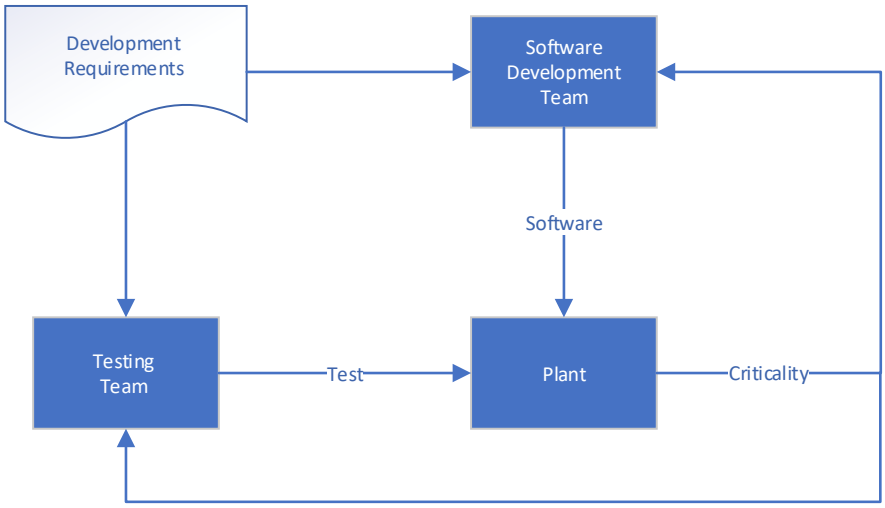

Figure 3.1: High-level abstraction of the automotive software development and validation process, showing the interaction between requirements, testing, development, integration, and feedback via criticality information.

From a systems perspective, the validation workflow can be interpreted as a *bilevel decision process* involving two actors with different local objectives but a shared long-term goal. At the upper level, testing decisions determine which system configurations are evaluated and at which stage of the integration process. At the lower level, software development decisions react to the revealed test outcomes by selectively remediating detected issues. The observed validation data-tickets, severity levels, timestamps, and metadata are direct outcomes of this sequential interaction. To formalize this perspective, let $\mathbf{c}$ denote a tested configuration, where $\mathbf{c} = (m, f, \ell, t, \mathbf{x})$ comprises the car model, software function, production line, integration time, and associated configuration metadata. Let $R(\mathbf{c})$ denote the expected risk associated with configuration $\mathbf{c}$. The quantity $R(\mathbf{c})$ summarizes both the probability of a ticket and the expected severity conditional on its occurrence, and thus represents the anticipated criticality of testing $\mathbf{c}$.
The component $t(\mathbf{c})$ denotes the integration time at which configuration $\mathbf{c}$ is tested. Time is observed as part of the configuration metadata and may correspond, for example, to an integration stage, release index, or calendar-based milestone. To reflect the well-established observation that late detection and remediation of defects

incur higher effective cost, we introduce a non-negative time-weighting function $w(t)$ with $w'(t) \geq 0$. The function $w(\cdot)$ is not required to follow a specific parametric form; it serves as an abstract representation of increasing process cost over integration time.
The cumulative process cost over a set of tested configurations $\mathcal{C}$ is then defined as

$$\mathcal{J} = \sum_{\mathbf{c} \in \mathcal{C}} w(t(\mathbf{c}))\ R(\mathbf{c}). \tag{3.1}$$

This expression aggregates the expected risk contributions of all tested configurations, weighted by the stage of the integration process at which the risk is revealed. Configurations with high expected risk contribute more strongly to the total cost when they are tested at later stages.
Within this abstraction, test prioritization can be understood as allocating limited testing resources to configurations with a high expected contribution to the cumulative process cost. By exposing high-risk configurations early, testing provides actionable information for subsequent development activities and reduces the likelihood that critical issues propagate into later integration stages. Software development decisions then act on the revealed information to mitigate the accumulated cost over time.
This interaction may be interpreted as a *sequential min–max process*, in the sense that testing decisions reveal risk while development decisions subsequently mitigate it. However, no equilibrium or adversarial game-theoretic formulation is assumed. The bilevel perspective is used solely as a conceptual abstraction to motivate risk-based test prioritization and to clarify the role of predictive uncertainty in the validation workflow.
A detailed illustration of the ticket creation and validation workflow underlying this abstraction is provided in Appendix A. The appendix outlines the end-to-end process through which development requirements are translated into test plans, test executions, documented faults, and subsequent remediation cycles.
While the full workflow contains several operational decision points (e.g., ticket rejection, re-opening, or reclassification), these details are not modeled explicitly in this thesis. Instead, the modeling approach deliberately abstracts from individual process branches and focuses on the resulting statistical regularities observable in the historical ticket database, namely the occurrence of tickets, their associated severity levels, and their temporal distribution.
The software artefact developed in this work operates on this historical database. Given a set of candidate system configurations, the artefact produces a ranked list of proposed tests with the objective of allocating limited testing resources to configurations with the highest expected contribution to the cumulative process cost defined in (3.1). By prioritizing configurations with high predicted risk at early integration stages, the artefact supports informed testing decisions and provides actionable feedback to software development. In the long term, this enables a reduction in the accumulation of critical issues at later stages of system integration.

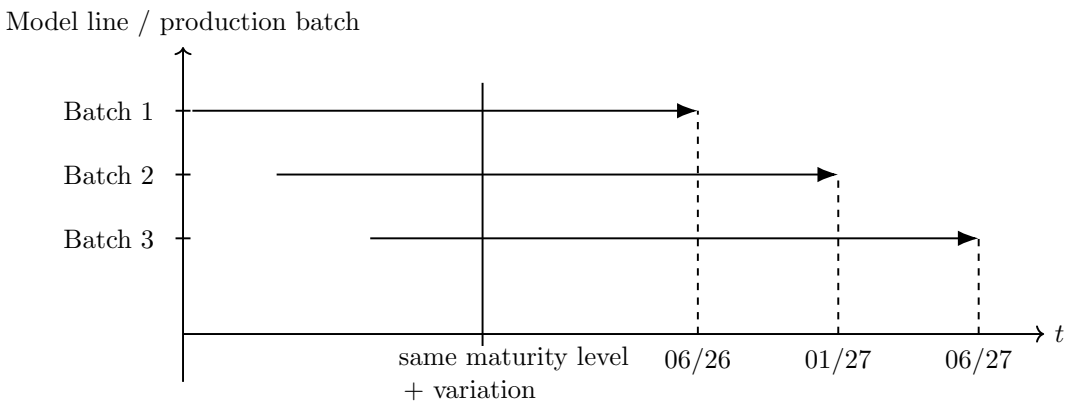

Figure 3.2: Three production batches of the same car model, produced in overlapping time intervals with a small temporal offset. Vertical dashed lines mark calendar milestones aligned with the batch endpoints.

Figure 3.2 illustrates this concept schematically. The horizontal arrows represent three production batches of the same car model that overlap in time but are initiated and completed at different integration stages. Batch 1 starts earliest and reaches its endpoint first, followed by Batch 2 and Batch 3, reflecting staggered but parallel development.

The vertical dashed lines indicate selected calendar or integration milestones and terminate exactly at the corresponding batch endpoints. At intermediate points along the time axis, batches may occupy similar maturity levels, as indicated by the vertical marker labeled "same maturity level + variation". This highlights that, although batches are temporally offset, they can share a comparable software state while still exhibiting batch-specific deviations.

An additional source of coupling arises from shared software updates. When a software release is deployed at a given integration time, the same software state is typically propagated across all active batches. Such events introduce synchronized changes across batches and further strengthen their interdependence. From a modeling perspective, this motivates the need to capture dependencies between batches explicitly, rather than treating them as independent validation streams.

3.2 Data Preprocessing and Dataset Construction

Building on the process abstraction in Section 3.1, this section translates the raw validation artifacts into modeling-ready datasets that align with the probabilistic variables introduced later in Sec. 3.3.

The raw validation data is heterogeneous and consists of categorical identifiers, timestamps, and sparsely populated metadata fields originating from multiple industrial data sources. To ensure statistical consistency, comparability across config-

urations, and robust probabilistic modeling, a structured preprocessing and dataset construction workflow is applied prior to model training.

Tabular representation and dependency structure. All validation data are represented in tabular form, where each row corresponds to a tested system configuration and each column represents either categorical identifiers (e.g., function code, car model, production batch), temporal information (e.g., integration step, ticket creation time), or configuration metadata (e.g., ECU type, architectural variant).
While the data are stored in a tabular format, individual observations are not independent in practice. Domain experts emphasize that validation outcomes may depend on previous test results within the same production batch, on earlier integration stages, or on related batches of the same car model. However, the exact dependency structure is only partially observable and cannot be reconstructed reliably from the available records. Consequently, these dependencies are not enforced explicitly during preprocessing, but are instead captured implicitly by the probabilistic model through learned relationships across observations and time.
This abstraction is consistent with established workflows in data-driven test prioritization, where tabular representations are used to enable systematic analysis and comparability across methods, while complex dependencies are addressed at the modeling stage rather than through handcrafted features [4].

Data cleaning and feature selection. The raw data are collected from the central validation database and processed in several steps. Duplicate records are removed, rows containing invalid or undefined values are discarded, and attributes that are not relevant for the modeling objective are excluded. After cleaning, each configuration is described by its software function, car model, production batch, integration step, ticket timestamp, severity label, and a set of metadata fields characterizing the system configuration.
Ticket severity is originally recorded on a discrete scale from 1 (lowest impact) to 7 (highest impact). The empirical distribution of severity levels is highly imbalanced, with several levels accounting for less than five percent of all tickets. Guided by expert recommendations and motivated by the downstream goal of risk-based ranking rather than fine-grained classification, severity levels are aggregated into three ordered groups: 1 (low), 2 (medium), and 3 (high) severity. This aggregation reduces statistical noise in sparsely populated classes while preserving the ordinal structure of severity levels required for meaningful risk-based ranking.

Construction of occurrence and severity datasets. The probabilistic modeling approach distinguishes explicitly between ticket occurrence and ticket severity. As a result, two related but structurally different datasets are constructed.
The severity dataset consists exclusively of configurations for which a ticket occurred and is labeled with the aggregated severity class. This dataset is obtained directly from the cleaned validation records.

The occurrence dataset represents both positive and negative outcomes, i.e., configurations where a ticket occurred ($T = 1$) and configurations where no ticket was observed ($T = 0$). Since the database records only failures, the absence of a ticket is interpreted, following expert guidance, as a successful test execution. Configurations without a ticket are therefore treated as negative samples, implicitly assuming that they were tested and behaved as expected. Unobserved or untested configurations are not modeled explicitly.
To construct negative samples, all observed (function, model, batch) tuples are considered, and for integration steps where no ticket is recorded, a synthetic observation with $T = 0$ is created. However, a naive application of this procedure across all integration steps leads to a severe class imbalance, with negative samples dominating the dataset and potentially exceeding 90% of all observations.

Hard negative sampling. To control class imbalance and concentrate learning on informative regions of the input space, a hard negative sampling strategy is employed for modeling ticket occurrence. Rather than generating negative samples uniformly across all integration steps, negative examples are selected such that they are close to observed failures in *integration progress* and configuration, but differ in outcome. This focuses learning on configurations for which ticket occurrence is plausible and avoids overwhelming the model with trivially safe cases.
Let $\mathcal{P}$ denote the set of observed ticket occurrences,

$$\mathcal{P} = \big\{\mathbf{c}_n = (m, f, \ell, i_n, \mathbf{x}_n) \;\big|\; T_n = 1\big\},$$

where m denotes the vehicle model, f the software function, ℓ the production or release line, $i_n \in \{1, \ldots, I\}$ the corresponding integration step within a configuration, and $\mathbf{x}_n \in \mathbb{R}^d$ the associated feature vector. The index $n = 1, \ldots, N$ enumerates individual configuration instances. Furthermore, let $\mathcal{N}$ denote the set of candidate negative configurations with $T = 0$ for the same (m, f, ℓ) tuple.
For each positive instance $\mathbf{c}_n \in \mathcal{P}$, negative samples are generated only from configurations whose integration step lies within a local neighborhood around i_n,

$$\mathcal{N}_n^{\text{hard}} = \big\{\mathbf{c} \in \mathcal{N} \;\big|\; |i(\mathbf{c}) - i_n| \leq \Delta\big\}, \tag{3.2}$$

where $\Delta > 0$ defines a small radius in the ordered integration-step space. The variable $i(\mathbf{c})$ denotes the integration step associated with configuration $\mathbf{c}$ and reflects ordinal progress within the validation process rather than wall-clock time. All negative samples used for training are drawn from the union $\mathcal{N}^{\text{hard}} = \bigcup_n \mathcal{N}_n^{\text{hard}}$.
This construction ensures that negative samples correspond to configurations that are structurally similar to known failures, sharing the same vehicle model, software function, and release line, while differing only slightly in integration step. As a result, the learning problem emphasizes configurations near the implicit decision boundary between ticket occurrence and non-occurrence, rather than regions where tickets are extremely unlikely.

From a probabilistic learning perspective, hard negative sampling concentrates training mass on samples with higher posterior uncertainty. Under a log-loss objective, these samples contribute most strongly to the expected loss,

$$\mathcal{L} = \mathbb{E}_{(\mathbf{c},T)}\big[-T \log p_{\text{ticket}}(\mathbf{c}) - (1-T)\log\big(1 - p_{\text{ticket}}(\mathbf{c})\big)\big], \tag{3.3}$$

because they correspond to cases where $p_{\text{ticket}}(\mathbf{c}) \approx 0.5$ rather than being trivially close to zero. Focusing on such samples improves discrimination performance and calibration, while reducing the effective imbalance between positive and negative samples.
Hard negative sampling is a well-established technique in unbalanced classification and contrastive learning. By emphasizing difficult negative examples that are close to positive instances in the feature or ordinal process space, it has been shown to improve both convergence stability and generalization performance [31–33].
In the present context, this strategy prevents uncontrolled growth of the synthetic occurrence dataset and mitigates the risk of introducing distributional bias.

3.3 Mathematical Modeling

Using the datasets constructed in Sec. 3.2, we now formalize a probabilistic model that mirrors the industrial process described in Sec. 3.1. The formulation explicitly defines key quantities (ticket probability, severity distribution, and expected risk), enabling their consistent reuse throughout the remainder of the thesis.
The objective is to model ticket occurrence and severity as observable manifestations of an underlying time-varying risk process. Automotive validation data typically exhibits hierarchical and cross-classified dependencies, as observations are indexed jointly by car model, software function, and production line. These groupings induce systematic similarities that cannot be captured reliably when treating each record independently. Consequently, we adopt a structured probabilistic formulation that accounts for (i) temporal variation, (ii) information sharing across aggregation levels, and (iii) probabilistic predictions required for risk-based prioritization.
The partial pooling principle follows the standard multilevel motivation: it enables statistically efficient information sharing in data-sparse groups while allowing data-rich groups to deviate when supported by evidence [34].

Index sets and configuration representation. To formalize the structure of the validation domain, we introduce three finite index sets: car models $\mathcal{M} = \{1, \dots, M\}$, software functions $\mathcal{F} = \{1, \dots, F\}$, and production lines (or production quarters) $\mathcal{L} = \{1, \dots, L\}$. A tested configuration at time t is represented as

$$\mathbf{c} := (m, f, \ell, t, \mathbf{x}),$$

where $m \in \mathcal{M}$, $f \in \mathcal{F}$, $\ell \in \mathcal{L}$, and $\mathbf{x} \in \mathbb{R}^d$ denotes observed metadata/features (e.g., derivative, integration level, ECU, drive type, and development time). This repre-

sentation is compatible with multiple analysis views that arise in practice. Specifically, we consider three nested aggregation levels ordered from the most specific to the most global perspective:

$$\begin{aligned} \text{Level 1 (model–function specific):} \quad & (m, f) \text{ fixed}, \quad \ell \in \mathcal{L}, \\ \text{Level 2 (model-specific):} \quad & m \text{ fixed}, \quad (f, \ell) \in \mathcal{F} \times \mathcal{L}, \\ \text{Level 3 (global):} \quad & (m, f, \ell) \in \mathcal{M} \times \mathcal{F} \times \mathcal{L}. \end{aligned} \tag{3.4}$$

Rather than fitting disconnected models for each aggregation level in (3.4), we capture all views within a single hierarchical model via partial pooling, which yields a coherent probabilistic treatment across levels [34].

Latent dynamic risk state. For each triple (m, f, ℓ) we introduce a latent scalar state $z_{m,f,\ell}(t) \in \mathbb{R}$ that represents the unobserved instability or maturity of the corresponding model–function–line combination at time t. The role of $z_{m,f,\ell}(t)$ is to summarize historical evidence and latent influences that are not explicitly represented by the observed covariates $\mathbf{x}$, while still affecting both the probability of observing a ticket and the severity of that ticket when it occurs. This perspective is consistent with standard latent-state and state-space modeling, where unobserved system characteristics influence observations indirectly and evolve over time [35, Ch. 8.1–8.3].
To capture temporal persistence, we assume first-order Markov dynamics in discrete time,

$$\begin{aligned} z_{m,f,\ell}(t) &= z_{m,f,\ell}(t-1) + \eta_{m,f,\ell}(t), \\ \eta_{m,f,\ell}(t) &\sim \mathcal{N}(0, \sigma^2_{m,f,\ell}). \end{aligned} \tag{3.5}$$

The first-order Markov property states that the future state depends on the past only through the immediate previous state, an assumption that underlies classical sequential latent-variable models and is widely used as a parsimonious summary of historical information [36]. The random-walk form in (3.5) is a standard building block in state-space models and can be interpreted as a temporally smooth yet adaptive evolution model: innovations $\eta_{m,f,\ell}(t)$ capture net changes induced by integration updates, regressions, fixes, or test-environment shifts that occur between time steps [37]. A zero-mean Gaussian innovation reflects the absence of a deterministic drift and provides a tractable model for the aggregate effect of many small, heterogeneous influences [37].

Coupling latent risk with observed covariates. We combine the latent state with observed configuration features via a linear predictor

$$\psi_{m,f,\ell}(t, \mathbf{x}) \;=\; z_{m,f,\ell}(t) + \mathbf{w}^\top \mathbf{x}, \tag{3.6}$$

where $\mathbf{w} \in \mathbb{R}^d$ models global covariate effects shared across groups. Importantly, $\mathbf{w}^\top \mathbf{x}$ is not assumed to be sufficient to explain ticket occurrence or severity in isolation. In early exploratory baselines, a purely linear "row-wise" mapping from

features to outcomes was unable to capture the relevant structure, indicating that predictive signal arises from interactions across groups and time rather than from isolated records. This observation motivates the introduction of $z_{m,f,\ell}(t)$ as a latent carrier of cross-row and temporal information beyond what is explicitly available in $\mathbf{x}$. The separation between (global) covariate effects and (group- and time-specific) latent structure follows standard statistical learning intuition: linear predictors offer interpretability but often require augmentation when the data exhibit complex heterogeneity and non-linear dependency patterns [16, Ch. 4.4].

Hierarchical information sharing via partial pooling. To formalize information transfer across production lines, functions, and models, we introduce a multi-level prior over group-specific parameters. Let $\theta_{m,f,\ell}$ denote parameters governing the typical behavior of the latent process for line ℓ within function f and model m (e.g., the initial state $z_{m,f,\ell}(0)$ or the volatility $\sigma^2_{m,f,\ell}$). We model these line-level parameters as deviations from their parent function-level parameters,

$$\theta_{m,f,\ell} \mid \theta_{m,f} \sim \mathcal{N}(\theta_{m,f}, \Sigma_L), \tag{3.7}$$

and assume function-level parameters deviate from model-level parameters,

$$\theta_{m,f} \mid \theta_m \sim \mathcal{N}(\theta_m, \Sigma_F), \tag{3.8}$$

while model-level parameters deviate from a global parameter,

$$\theta_m \mid \theta_0 \sim \mathcal{N}(\theta_0, \Sigma_M), \tag{3.9}$$

with a weakly informative global prior

$$\theta_0 \sim \mathcal{N}(0, \Sigma_0). \tag{3.10}$$

Equations (3.7)–(3.10) implement partial pooling: groups with few observations are shrunk toward higher-level estimates, whereas groups with strong evidence can deviate more strongly, yielding stable inference in cross-classified and imbalanced data regimes [34]. The Gaussian assumptions are not intended as literal ground truth; they provide a pragmatic regularization mechanism compatible with the Gaussian innovation model in (3.5) and are widely used in hierarchical Bayesian modeling due to their analytical convenience and robustness [34].
The resulting formulation induces three application-aligned assumptions: (i) temporal smoothness with adaptability (random-walk dynamics), (ii) conditional exchangeability within hierarchical levels (justifying pooling), and (iii) structured heterogeneity, where each (m, f, ℓ) retains an individual trajectory while borrowing strength from parent levels [34].

3.3.1 Probabilistic Model for Ticket Occurrence, Severity, and Risk

The observation process in automotive validation exhibits a fundamental asymmetry: the absence of a ticket indicates a successful test, whereas severity information

is only defined if a ticket occurs. Modeling severity as an unconditional multi-class outcome for all configurations would therefore conflate "no ticket" with "low severity" and violate the semantics of the validation process. To respect this structure, we explicitly decouple (i) the occurrence of a ticket and (ii) the severity of a ticket conditional on its occurrence, and subsequently derive a unified probabilistic representation suitable for risk-based prioritization.
Let $\mathbf{c} = (m, f, \ell, t, \mathbf{x})$ denote the configuration descriptor, and define two observable random variables

$$T \in \{0, 1\}, \qquad S \in \{1, 2, 3\},$$

where $T = 1$ denotes ticket occurrence and S denotes severity in the case that a ticket occurs. Risk-based decision making requires access to the full predictive distributions $p(T = 1 \mid \mathbf{c})$ and $p(S = k \mid T = 1, \mathbf{c})$, rather than only point predictions, because expected-risk measures depend on probability mass across all severity levels.

Ticket occurrence and severity models. Given the linear predictor $\psi(\mathbf{c})$ defined in (3.6), ticket occurrence is modeled as

$$P(T = 1 \mid \mathbf{c}) = \sigma(\psi(\mathbf{c})), \tag{3.11}$$

where $\sigma(\cdot)$ denotes the logistic sigmoid. The quantity $P(T = 1 \mid \mathbf{c})$ is interpreted as the posterior predictive probability of ticket occurrence and is obtained in practice from a PFN trained on historical ticket data.
Conditional on ticket occurrence, severity is modeled as a categorical distribution over severity levels $k \in \{1, 2, 3\}$,

$$P(S = k \mid T = 1, \mathbf{c}) = \mathrm{softmax}_k\big(\mathbf{a}_k^\top \phi(\psi(\mathbf{c}))\big), \qquad k \in \{1, 2, 3\}, \tag{3.12}$$

where $\phi(\cdot)$ is a (possibly identity) link function. This formulation ensures that severity probabilities are only defined when a ticket occurs and corresponds to standard multinomial modeling of categorical outcomes [16, Ch. 4.4]. Again, the softmax outputs are interpreted as posterior predictive probabilities estimated by a PFN trained on severity-labeled data restricted to the subset $T = 1$.

Decoupled probabilistic structure. For clarity, we introduce the shorthand

$$p_{\text{ticket}}(\mathbf{c}) = P(T = 1 \mid \mathbf{c}), \qquad s_k(\mathbf{c}) = P(S = k \mid T = 1, \mathbf{c}),$$

with $\sum_{k=1}^{3} s_k(\mathbf{c}) = 1$. This yields an explicit factorization of the validation outcome into

- a binary model for ticket occurrence defined in (3.11), and
- a conditional three-class model for severity given occurrence defined in (3.12).

The decoupling reflects the semantics of the validation process and avoids conflating the absence of a ticket with low severity.

Joint distribution over ticket absence and severity. To obtain a single probabilistic model over both successful tests and all severity levels, we define a combined random variable

$$C \in \{0, 1, 2, 3\},$$

where $C = 0$ denotes no ticket and $C = k$ denotes a ticket of severity k. Using the product rule of conditional probability, the joint distribution is given by

$$\begin{aligned} P(C = 0 \mid \mathbf{c}) &= P(T = 0 \mid \mathbf{c}) = 1 - p_{\text{ticket}}(\mathbf{c}), && (3.13) \\ P(C = k \mid \mathbf{c}) &= P(T = 1, S = k \mid \mathbf{c}) \\ &= P(T = 1 \mid \mathbf{c})\, P(S = k \mid T = 1, \mathbf{c}) \\ &= p_{\text{ticket}}(\mathbf{c})\, s_k(\mathbf{c}), \qquad k \in \{1, 2, 3\}. && (3.14) \end{aligned}$$

[38, Sec. 2.1–2.2].

Normalization. The resulting distribution over C is properly normalized. Substituting (3.13) and (3.14) yields

$$\begin{aligned} \sum_{c=0}^{3} P(C = c \mid \mathbf{c}) &= (1 - p_{\text{ticket}}(\mathbf{c})) + p_{\text{ticket}}(\mathbf{c}) \sum_{k=1}^{3} s_k(\mathbf{c}) \\ &= (1 - p_{\text{ticket}}(\mathbf{c})) + p_{\text{ticket}}(\mathbf{c}) = 1. && (3.15) \end{aligned}$$

Thus, $P(C \mid \mathbf{c})$ defines a valid four-class probability distribution.

Probability of any ticket. The marginal probability that any ticket occurs for configuration $\mathbf{c}$ follows directly from (3.11):

$$P(\text{ticket occurs} \mid \mathbf{c}) = P(T = 1 \mid \mathbf{c}) = p_{\text{ticket}}(\mathbf{c}). \tag{3.16}$$

Expected risk. While severity represents a conditional criticality level, the quantity of interest for validation planning is risk, which must account for both the likelihood of a ticket and its potential impact. We therefore define the risk of configuration $\mathbf{c}$ as the conditional expectation of C:

$$R(\mathbf{c}) = \mathbb{E}[C \mid \mathbf{c}] = \sum_{c=0}^{3} c\, P(C = c \mid \mathbf{c}). \tag{3.17}$$

Using the joint distribution from (3.13) and (3.14), we obtain

$$\begin{aligned} R(\mathbf{c}) &= \sum_{k=1}^{3} k\, P(C = k \mid \mathbf{c}) \\ &= \sum_{k=1}^{3} k\, p_{\text{ticket}}(\mathbf{c})\, s_k(\mathbf{c}) \\ &= p_{\text{ticket}}(\mathbf{c}) \sum_{k=1}^{3} k\, s_k(\mathbf{c}). && (3.18) \end{aligned}$$

[39].

3.3.2 Prior Classifier Baseline (Empirical Prior Model)

Before introducing feature-aware models, we define a principled *prior-only* baseline that deliberately ignores the configuration descriptor $\mathbf{c}$. Its role is twofold: (i) it provides a transparent lower bound for probabilistic prediction under proper scoring rules, and (ii) it isolates the contribution of feature information by testing whether any model truly learns beyond global label frequencies.
From a probabilistic learning perspective, this baseline corresponds to predicting with the *empirical class prior* induced by the training set, i.e., the maximum likelihood estimate of the Bernoulli (or categorical) parameters when the predictor is constrained to be constant [40, Ch. 3–4]. In other words, it minimizes the empirical log-loss among all models that predict a constant probability independent of the configuration.

Ticket occurrence prior. Let $\mathcal{D}_A = \{(\mathbf{c}_i, T_i)\}_{i=1}^{N_A}$ denote the occurrence dataset. The prior classifier predicts the same probability for all configurations,

$$\hat{p}^{\text{prior}}_{\text{ticket}}(\mathbf{c}) := \hat{p}(T = 1) = \frac{1}{N_A} \sum_{i=1}^{N_A} \mathbb{I}[T_i = 1], \tag{3.19}$$

which is the empirical Bernoulli parameter estimate [40, Sec. 3.2]. This baseline corresponds to a context-agnostic predictor that relies solely on global historical ticket frequencies, without incorporating configuration-specific information.

Severity prior conditional on ticket occurrence. Let $\mathcal{D}_B = \{(\mathbf{c}_j, S_j)\}_{j=1}^{N_B}$ denote the severity dataset restricted to $T = 1$. The prior severity distribution is

$$\hat{p}^{\text{prior}}_{\text{sev}}(k \mid T = 1, \mathbf{c}) := \hat{p}(S = k \mid T = 1) = \frac{1}{N_B} \sum_{j=1}^{N_B} \mathbb{I}[S_j = k], \qquad k \in \{1, 2, 3\}, \tag{3.20}$$

which is the empirical categorical parameter estimate under an i.i.d. multinomial model [40, Sec. 3.3].

Risk induced by the prior baseline. Combining (3.18), (3.19), and (3.20) yields a constant (configuration-independent) risk:

$$R^{\text{prior}}(\mathbf{c}) = \hat{p}(T = 1) \sum_{k=1}^{3} k\, \hat{p}(S = k \mid T = 1), \tag{3.21}$$

so this baseline cannot produce a meaningful ranking beyond ties. In practice, it is therefore mainly used as a calibration and sanity reference: any feature-aware model should outperform it under log-loss and should induce a non-trivial risk ordering.

3.3.3 Random Forest Baseline (Nonlinear Feature-Based Ensemble)

To obtain a strong classical baseline for heterogeneous tabular data, we employ Random Forests (RF), which are ensemble predictors built from randomized decision trees and are known to provide robust performance with minimal feature engineering [18]. RFs are particularly relevant in the present application because (i) they capture nonlinear interactions among categorical and lag features, (ii) they remain stable under mixed-scale inputs and sparsity, and (iii) they produce probabilistic class estimates that can be inserted directly into the risk decomposition in (3.18).

Ensemble construction. Given a training set $\mathcal{D} = \{(\mathbf{c}_i, y_i)\}_{i=1}^N$, a Random Forest consists of M trees $\{h_m\}_{m=1}^M$ trained on bootstrap samples of $\mathcal{D}$. At each split, a random subset of features is considered, inducing decorrelation between trees and reducing variance [18].

Entropy-Based Split Criterion and Model Fitting. In this work, Random Forest models are fitted using entropy-based recursive partitioning. Each individual decision tree is trained on a bootstrap sample of the configuration dataset, while at every split only a random subset of features is considered. This ensemble strategy reduces variance and improves generalization performance [18].
Within each tree, Shannon entropy is used as the impurity measure. For a node containing a subset of configurations with empirical class probabilities $p_k = \mathbb{P}(Y = k)$, the entropy is defined as

$$H = -\sum_{k=1}^{K} p_k \log p_k, \tag{3.22}$$

where K denotes the number of classes [16, 17]. This formulation applies to both binary ticket occurrence ($K = 2$) and multi-class severity prediction ($K = 3$). In the binary case, the expression reduces to $H(p) = -p \log p - (1-p)\log(1-p)$, where $p = \mathbb{P}(T = 1)$.
During training, candidate splits of configuration features (e.g., integration stage, vehicle model, software variant, or historical indicators) are evaluated according to their expected entropy reduction, also known as information gain:

$$\Delta H = H(\text{parent}) - \sum_{j \in \{\text{left,right}\}} \frac{n_j}{n} H(\text{child}_j), \tag{3.23}$$

where n denotes the number of configurations in the parent node and n_j the number of configurations in child node j. The split that maximizes ΔH is selected [16].
This greedy procedure is applied recursively until predefined stopping criteria are met (e.g., minimum node size or maximum depth). As a result, the configuration space is partitioned into regions with increasingly homogeneous class distributions. The final Random Forest prediction is obtained by averaging class probabilities

across all trees, yielding a robust estimate of $\mathbb{P}(Y \mid c)$, where Y denotes the respective target variable. In the case of ticket occurrence, $Y = T \in \{0, 1\}$, whereas for severity prediction, $Y = S \in \{1, 2, 3\}$.

RF probabilities for ticket occurrence (Model A). For binary ticket occurrence $T \in \{0, 1\}$, each tree h_m produces a class label, and the forest probability is defined as the average vote:

$$\hat{p}^{\mathrm{RF}}_{\mathrm{ticket}}(\mathbf{c}) = \frac{1}{M} \sum_{m=1}^{M} \mathbb{I}\big[h_m(\mathbf{c}) = 1\big]. \tag{3.24}$$

Equivalently, when trees output leaf class frequencies, (3.24) corresponds to averaging per-tree probability estimates [18].

RF probabilities for conditional severity (Model B). For severity $S \in \{1, 2, 3\}$ trained only on ticketed cases ($T = 1$), each tree yields a categorical distribution via leaf frequencies. Denote by $\hat{p}_m(S = k \mid T = 1, \mathbf{c})$ the per-tree probability returned for class k. The RF severity estimate is

$$\hat{p}^{\mathrm{RF}}_{\mathrm{sev}}(k \mid T = 1, \mathbf{c}) = \frac{1}{M} \sum_{m=1}^{M} \hat{p}_m(S = k \mid T = 1, \mathbf{c}), \qquad k \in \{1, 2, 3\}. \tag{3.25}$$

Risk computation with RF. The RF-induced expected risk follows directly by substituting (3.24) and (3.25) into the factorized risk definition in Eq. (3.18):

$$R^{\mathrm{RF}}(\mathbf{c}) = \hat{p}^{\mathrm{RF}}_{\mathrm{ticket}}(\mathbf{c}) \sum_{k=1}^{3} k\, \hat{p}^{\mathrm{RF}}_{\mathrm{sev}}(k \mid T = 1, \mathbf{c}). \tag{3.26}$$

This yields a nonlinear yet fully probabilistic baseline for risk ranking.

3.3.4 Prior-Data Fitted Networks (PFN) for Hierarchical Risk Inference

While the prior and RF baselines provide useful reference points, the central requirement of this thesis is *probabilistic prediction under structured data scarcity*: the effective sample size per (m, f, ℓ) cell may be small, labels are imbalanced, and the data are cross-classified and time-dependent. Prior-Data Fitted Networks (PFNs) address this regime by learning, during pretraining, to approximate Bayesian posterior predictive inference for a broad family of data-generating processes, enabling sample-efficient probabilistic predictions at downstream time [20, 21, 23].

Posterior predictive objective. Let $\mathcal{D} = \{(\mathbf{c}_i, y_i)\}_{i=1}^{n}$ be a task-specific context set (training examples), and let $\mathbf{c}_*$ denote a query configuration. Bayesian prediction is based on the posterior predictive distribution

$$p(y_* \mid \mathbf{c}_*, \mathcal{D}) = \int p(y_* \mid \mathbf{c}_*, \theta)\, p(\theta \mid \mathcal{D})\, d\theta, \tag{3.27}$$

where θ denotes latent task parameters. PFNs are trained to approximate (3.27) directly with a neural function f_ϕ that takes the dataset context and the query as input,

$$\hat{p}(y_* \mid \mathbf{c}_*, \mathcal{D}) = f_\phi(\mathcal{D}, \mathbf{c}_*), \tag{3.28}$$

where ϕ are learned meta-parameters obtained by pretraining on many synthetic tasks drawn from a prior over data-generating processes [23]. In this sense, PFNs can be interpreted as amortizing Bayesian inference: inference for a new task is performed by a single forward pass conditioned on the observed context set $\mathcal{D}$.

Application to the two-stage risk decomposition. In the present setting, we instantiate (3.28) twice:

$$\hat{p}^{\text{PFN}}_{\text{ticket}}(\mathbf{c}) = f_{\phi_A}(\mathcal{D}_A, \mathbf{c}) \;\approx\; P(T = 1 \mid \mathbf{c}), \tag{3.29}$$

$$\hat{p}^{\text{PFN}}_{\text{sev}}(k \mid T = 1, \mathbf{c}) = f_{\phi_B}(\mathcal{D}_B, \mathbf{c})_k \;\approx\; P(S = k \mid T = 1, \mathbf{c}), \qquad k \in \{1, 2, 3\}, \tag{3.30}$$

where $\mathcal{D}_A$ is the occurrence dataset (including hard negatives) and $\mathcal{D}_B$ is the severity dataset restricted to $T = 1$. The risk score then follows exactly the factorization in Eq. (3.18):

$$R^{\text{PFN}}(\mathbf{c}) = \hat{p}^{\text{PFN}}_{\text{ticket}}(\mathbf{c}) \sum_{k=1}^{3} k\, \hat{p}^{\text{PFN}}_{\text{sev}}(k \mid T = 1, \mathbf{c}). \tag{3.31}$$

Cross-architecture domain adaptation via fine-tuning.
In addition to pure in-context inference, we investigate a domain-adapted variant of both Model A and Model B. For this purpose, we perform supervised fine-tuning on validation data from a previous software architecture generation. This historical dataset is not used for evaluation. It serves only to adapt the PFN meta-parameters before inference on the target architecture.
PFNs approximate Bayesian posterior predictive inference via amortized meta-learning,

$$f_\phi(\mathcal{D}, x_*) \approx p(y_* \mid x_*, \mathcal{D}),$$

as described in [20, 21, 23]. Fine-tuning adapts the meta-parameters ϕ by minimizing the negative log-likelihood on a historical dataset $\mathcal{D}_{\text{hist}}$. The dataset is repeatedly split into a context set $\mathcal{D}_{\text{ctx}}$ and a query set $\mathcal{D}_{\text{qry}}$. The model receives the context set as conditioning information and predicts the labels of the query samples. The loss for one split is

$$\mathcal{L}_{\text{FT}}(\phi) = -\sum_{(x_i, y_i) \in \mathcal{D}_{\text{qry}}} \log p_\phi(y_i \mid x_i, \mathcal{D}_{\text{ctx}}). \tag{3.32}$$

For Model A (ticket occurrence, binary classification), the targets are $y_i = T_i \in \{0, 1\}$, and the loss corresponds to conditional binary cross-entropy. For Model B (severity classification), the targets are $y_i = S_i \in \{1, 2, 3\}$ (restricted to $T_i = 1$), and the loss corresponds to conditional multi-class cross-entropy.

This objective preserves the PFN principle: the model is not trained to predict labels independently, but to improve its ability to perform posterior predictive inference conditioned on a dataset. Fine-tuning therefore adapts the implicit prior encoded in ϕ to the automotive validation domain, while keeping the inference mechanism unchanged.
From a Bayesian perspective, this corresponds to shifting the implicit task prior toward the historical architecture domain, analogous to hierarchical prior updating in multilevel Bayesian models [41]. The underlying assumption is that defect mechanisms and interaction structures show partial stability across architecture generations. Under such structural similarity, domain adaptation can improve predictive performance [42, 43].
After fine-tuning, inference on the target architecture remains purely in-context:

$$\hat{p}(y_* \mid x_*, \mathcal{D}_{\text{target}}) = f_{\phi'}(\mathcal{D}_{\text{target}}, x_*).$$

3.3.5 Hybrid Risk Model via Model Specialization

The probabilistic decomposition introduced in Section 3.3.1 separates ticket occurrence and conditional severity into two structurally different prediction tasks. Since these tasks differ in label structure (binary vs. ordinal multi-class) and statistical behavior, a single model class does not necessarily perform optimally for both.
As shown in Chapter 4, Random Forest achieves superior recall and stable discrimination performance for binary ticket occurrence, whereas PFN-based models provide stronger multi-class severity estimation, particularly with respect to ordinal accuracy and probabilistic calibration [18, 23].
This complementary behavior motivates a hybrid strategy: occurrence probability is estimated using Random Forest, while conditional severity probabilities are obtained from PFN. The two components are then combined within the same expected-risk formulation defined in Eq. (3.18).
Let

$$p_{\text{RF}}(c) := \hat{p}^{\text{RF}}_{\text{ticket}}(c)$$

denote the Random Forest estimate of $\mathbb{P}(T = 1 \mid c)$ (cf. Eq. (3.22)), and let

$$s_k^{\text{PFN}}(c) := \hat{p}^{\text{PFN}}_{\text{sev}}(k \mid T = 1, c)$$

denote the PFN estimate of $\mathbb{P}(S = k \mid T = 1, c)$ (cf. Eq. (3.28)).
Using the same factorization as in Eqs. (3.13)-3.14), the hybrid joint distribution is

$$\mathbb{P}(C = 0 \mid c) = 1 - p_{\text{RF}}(c), \tag{3.33}$$

$$\mathbb{P}(C = k \mid c) = p_{\text{RF}}(c)\, s_k^{\text{PFN}}(c), \quad k \in \{1, 2, 3\}. \tag{3.34}$$

Substituting into Eq. (3.18) yields the hybrid expected risk

$$R_{\text{hybrid}}(c) = p_{\text{RF}}(c) \sum_{k=1}^{3} k\, s_k^{\text{PFN}}(c). \tag{3.35}$$

Thus, the hybrid model preserves the theoretical risk definition while exploiting task-specific strengths of the respective estimators.

3.4 Implementation

The proposed probabilistic modeling framework is implemented on a cloud-based infrastructure to ensure computational scalability and reproducibility. All experiments are conducted on Amazon Web Services (AWS) using the SageMaker platform. Model development and experimentation are performed on a GPU-enabled notebook environment providing 16 vCPUs, 64 GB RAM, and an NVIDIA T4 GPU with 16 GB GPU memory, enabling accelerated inference and fine-tuning of transformer-based foundation models. All computations involving neural components are executed on a CUDA device to leverage GPU parallelization.

Figure 3.3 illustrates the overall implementation architecture. The pipeline follows the logical structure derived in the previous sections and mirrors the probabilistic decomposition of ticket occurrence and severity.

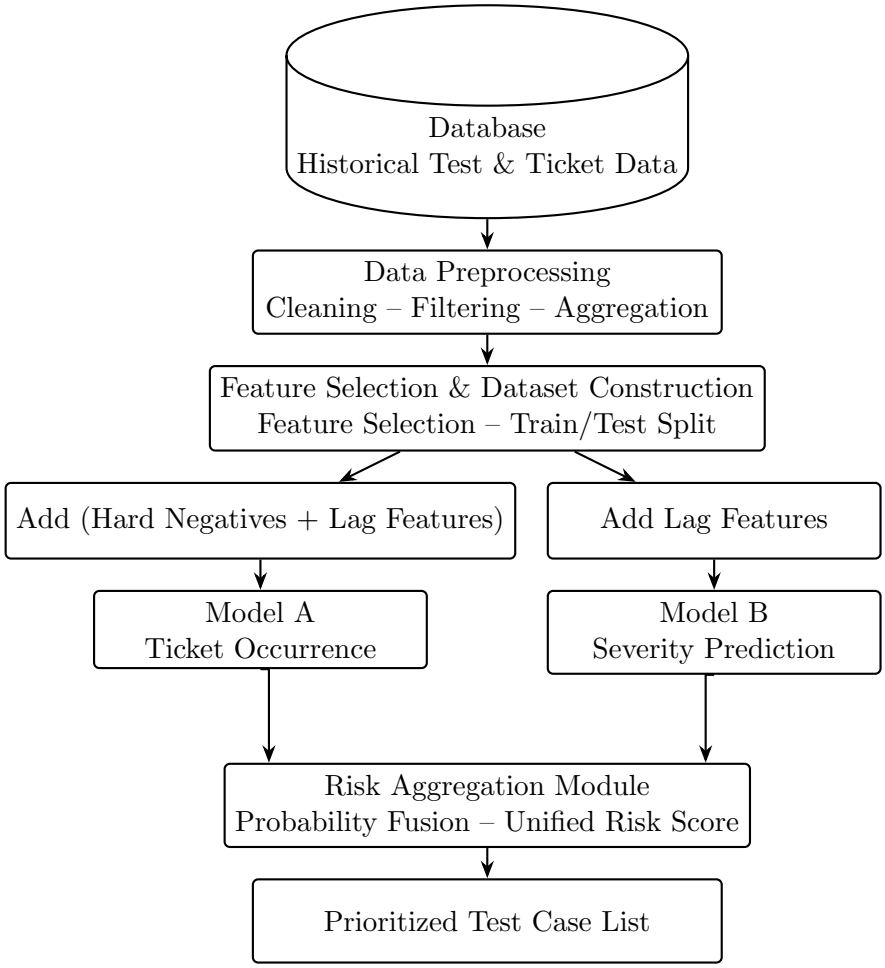

Figure 3.3: High-level architecture of the proposed two-stage data-driven prioritization framework.

Data ingestion and preprocessing. Historical validation data are retrieved from the central database and transformed into structured tabular form. The preprocessing stage implements the cleaning, filtering, aggregation, and dataset construction steps described in Sec 3.2. This includes severity aggregation, hard negative sampling for the occurrence dataset, and the construction of temporally ordered training and test splits. Feature selection is performed to retain configuration descriptors, integration step information, and relevant metadata required by the model.

Parallel modeling stage. Following preprocessing, two datasets are constructed in parallel.
Model A is trained to estimate the posterior predictive probability of ticket occurrence,

$$p_{\text{ticket}}(c) = P(T = 1 \mid c),$$

using the occurrence dataset augmented with hard negative samples. Lag features are incorporated to provide temporal context.
Model B is trained on the severity dataset restricted to $T = 1$, estimating the conditional severity distribution

$$s_k(c) = P(S = k \mid T = 1, c).$$

Both models are implemented using the tabular foundation model *TabPFN*, which is based on the Prior-Data Fitted Network (PFN) framework described in Sec. 3.3. TabPFN approximates the posterior predictive distribution directly through in-context learning (ICL), avoiding explicit gradient-based optimization for each downstream task.

Baseline models. In addition to the PFN-based models, two baseline approaches are implemented under the same preprocessing and dataset construction pipeline. Both baselines produce probabilistic outputs that are inserted into the identical risk aggregation module for fair evaluation.

Foundation model configuration. In the baseline configuration, both Model A and Model B are applied purely in in-context learning mode without fine-tuning. The entire downstream training dataset is provided as context to the pre-trained TabPFN model, which produces probabilistic predictions for unseen configurations. In a second experimental setting, the foundation model is fine-tuned on historical data from an older software architecture generation. After fine-tuning, in-context learning is again performed on the target dataset. This two-stage procedure allows us to evaluate whether domain adaptation through fine-tuning improves predictive performance relative to vanilla ICL, while preserving the probabilistic interpretation of the PFN outputs.

Risk aggregation and ranking. The outputs of Model A and Model B are combined in a dedicated risk aggregation module. For each configuration c, the unified risk score is computed as

$$R(c) = p_{\text{ticket}}(c) \sum_{k=1}^{3} k\, s_k(c),$$

consistent with (3.17). The resulting scalar risk value serves as the ranking criterion for prioritization. Configurations are sorted in descending order of $R(c)$, yielding a prioritized test case list.

Execution flow. The full implementation pipeline therefore proceeds sequentially as: database extraction, preprocessing, parallel PFN inference, probability fusion,

risk ranking. The modular structure enables independent evaluation of the occurrence and severity components while maintaining a coherent probabilistic interpretation aligned with the mathematical model.
This setup ensures compatibility with PFN-based Bayesian inference, supports uncertainty-aware decision making, and remains scalable to industrial-scale validation datasets through GPU acceleration.

3.5 Evaluation Metrics

This thesis evaluates a two-stage probabilistic modeling pipeline. Model A predicts ticket occurrence $T \in \{0, 1\}$, while Model B predicts the ordered severity $S \in \{1, 2, 3\}$ conditional on $T = 1$. The resulting probabilities are subsequently aggregated into an expected-risk score $\hat{R}$ that induces a prioritization (ranking) over candidate tests.

Model A (ticket occurrence, binary). Let $\mathcal{D}_A = \{(c_i, T_i)\}_{i=1}^{N_A}$ denote the evaluation set for ticket occurrence, and let $\hat{p}_i := \hat{p}_{\text{ticket}}(c_i)$ be the predicted probability of a ticket. For threshold-based metrics we use the induced hard prediction $\hat{T}_i := \mathbb{I}[\hat{p}_i \geq \tau]$ for a chosen threshold $\tau \in (0, 1)$.

Accuracy. Accuracy is the fraction of correctly classified instances:

$$\text{Acc}_A = \frac{1}{N_A} \sum_{i=1}^{N_A} \mathbb{I}[\hat{T}_i = T_i]. \tag{3.36}$$

Each term $\mathbb{I}[\hat{T}_i = T_i]$ equals 1 if the predicted class matches the true class and 0 otherwise. The metric therefore computes the proportion of correct predictions among all N_A instances. It provides an intuitive, global indicator of correctness, but can be misleading under strong class imbalance; it is therefore complemented by class-specific metrics such as recall [44].

Recall (sensitivity) for the positive class. Recall measures the proportion of true ticket occurrences that are correctly identified by the model. Using the evaluation set $\mathcal{D}_A$, it can be written as

$$\text{Rec}_A = \frac{\sum_{i=1}^{N_A} \mathbb{I}[\hat{T}_i = 1 \wedge T_i = 1]}{\sum_{i=1}^{N_A} \mathbb{I}[T_i = 1]}. \tag{3.37}$$

The numerator counts correctly detected tickets (true positives), while the denominator equals the total number of actual tickets in the evaluation set. Thus, recall corresponds to the conditional probability $\mathbb{P}(\hat{T} = 1 \mid T = 1)$. It is particularly important when missing true tickets (false negatives) is considered more critical than producing false alarms [44].

Logarithmic loss (log loss). Since the predicted probability $\hat{p}_{\text{ticket}}(c)$ is later used for risk aggregation, we evaluate probabilistic quality using the (negative) logarithmic score, i.e. log loss:

$$\mathcal{L}_A^{\log} = -\frac{1}{N_A} \sum_{i=1}^{N_A} \left(T_i \log(\hat{p}_i) + (1 - T_i) \log(1 - \hat{p}_i) \right). \tag{3.38}$$

For each instance, the term inside the sum selects $\log(\hat{p}_i)$ if $T_i = 1$ and $\log(1 - \hat{p}_i)$ if $T_i = 0$. The metric therefore penalizes confident but wrong predictions strongly, while confident and correct predictions incur only a small loss. Log loss evaluates the full predicted probability distribution and is a strictly proper scoring rule, meaning it is minimized when the predicted probabilities match the true data-generating distribution [45].

Model B (severity, three ordered classes). Let $\mathcal{D}_B = \{(c_i, S_i)\}_{i=1}^{N_B}$ denote the evaluation set restricted to ticketed cases ($T = 1$), with ordered labels $S_i \in \{1, 2, 3\}$. Here, N_B denotes the number of ticketed instances. Let $\hat{\mathbf{p}}_i := (\hat{p}_{i1}, \hat{p}_{i2}, \hat{p}_{i3})$ be the predicted class-probability vector with $\hat{p}_{ik} := \mathbb{P}(S = k \mid T = 1, c_i)$ and $\sum_{k=1}^{3} \hat{p}_{ik} = 1$. Thus, $\hat{p}_{ik}$ represents the model's confidence that configuration c_i belongs to severity class k.

Accuracy. Using the hard prediction $\hat{S}_i := \arg\max_{k \in \{1,2,3\}} \hat{p}_{ik}$, which selects the class with highest predicted probability, severity accuracy is

$$\text{Acc}_B = \frac{1}{N_B} \sum_{i=1}^{N_B} \mathbb{I}[\hat{S}_i = S_i]. \tag{3.39}$$

This metric measures the proportion of exactly correct severity predictions. All misclassifications are treated equally, regardless of their ordinal distance.

Multiclass log loss. To assess the full predictive distribution over severities we use the multiclass log loss:

$$\mathcal{L}_B^{\log} = -\frac{1}{N_B} \sum_{i=1}^{N_B} \log\big(\hat{p}_{i,S_i}\big). \tag{3.40}$$

Here, $\hat{p}_{i,S_i}$ denotes the predicted probability assigned to the true class S_i. The metric therefore penalizes low assigned probability to the correct class. As in the binary case, it evaluates calibration and sharpness simultaneously and constitutes a proper scoring rule for probability vectors [45].

Mean absolute error (MAE) for ordinal misclassification. Because severity is ordinal, we additionally measure the magnitude of misclassification. A standard MAE formulation for ordinal classification is expressed via the confusion matrix and penalizes errors proportionally to the label distance [46]:

$$\mathrm{MAE}_B = \frac{1}{N_B} \sum_{i=1}^{3} \sum_{j=1}^{3} C_{ij} \, |i - j|, \tag{3.41}$$

where C_{ij} counts instances of true class j predicted as class i. The term $|i - j|$ represents the ordinal distance between predicted and true class. Hence, confusing class 1 with 2 contributes less to the error than confusing class 1 with 3. The metric therefore reflects the average deviation in severity levels.

Expected risk metrics (ranking-based evaluation). For each configuration c_i in a ranking set $\mathcal{D}_R = \{(c_i, r_i)\}_{i=1}^{N_R}$ we define: (i) a predicted risk $\hat{R}_i := \hat{R}(c_i)$, and (ii) a realized risk $r_i \in \{0, 1, 2, 3\}$, where $r_i = 0$ if no ticket occurred and $r_i = S_i$ if a ticket occurred with severity S_i. The number of ranked items is denoted by N_R. Let π be a permutation of $\{1, \ldots, N_R\}$ that sorts items in *descending* order of predicted risk: $\hat{R}_{\pi(1)} \geq \hat{R}_{\pi(2)} \geq \cdots \geq \hat{R}_{\pi(N_R)}$. Thus, $\pi(1)$ corresponds to the highest-risk configuration.

Severity-weighted cumulative gain. Cumulative gains analysis evaluates how much of the target mass is captured in the top fraction of a ranked list [44]. We use a severity-weighted version:

$$\mathrm{CG}(\alpha) = \frac{\sum_{j=1}^{\lceil \alpha N_R \rceil} r_{\pi(j)}}{\sum_{i=1}^{N_R} r_i}, \qquad \alpha \in (0, 1]. \tag{3.42}$$

The numerator sums the realized severity of the top-ranked $\lceil \alpha N_R \rceil$ configurations, while the denominator represents the total realized severity across the entire dataset. Hence, $\mathrm{CG}(\alpha)$ measures which fraction of total severity is captured within the top α proportion of the ranking.

Recall@K for critical tickets (coverage at the top). To quantify how many truly critical items are prioritized early, we define a critical set $\mathcal{H} := \{i \mid r_i = 3\}$ and report recall at a fixed budget $K = \lfloor \alpha N_R \rfloor$:

$$\mathrm{Recall@}K = \frac{|\mathcal{H} \cap \{\pi(1), \ldots, \pi(K)\}|}{|\mathcal{H}|}. \tag{3.43}$$

The numerator counts how many truly critical configurations appear within the top-K ranked items, while the denominator is the total number of critical configurations. The metric therefore measures the fraction of the most severe tickets that are discovered within the available inspection budget.

Chapter 4

Setup and Results

> "The truth is obtained like gold, not by letting it grow bigger, but by washing off from it everything that isn't gold."
>
> — Leo Tolstoy [47]

4.1 Experimental Setup

This section describes the complete experimental setup, including dataset construction, temporal splitting strategy, and model training procedure. The objective is to ensure reproducibility of all reported results and a consistent evaluation protocol across models.

Dataset Partitioning and Scope. The original validation database is first divided into two disjoint subsets corresponding to a legacy software architecture and a new architecture generation. The separation is performed along the temporal feature `ticket_creation_time` such that no time overlap exists between both datasets. This prevents information leakage across architecture generations and ensures that historical tickets from the legacy system do not influence the evaluation on the new architecture. For both architectures, the analysis is restricted to the three most frequently documented car models. Within each selected model, the 200 most frequently occurring software functions are retained. This filtering step removes sparsely documented configurations and avoids unstable statistical behavior caused by extremely small sample sizes. The resulting dataset therefore focuses on configurations with sufficient historical evidence while preserving industrial relevance.

Construction of Model A and Model B Datasets. Two task-specific datasets are constructed according to the probabilistic decomposition introduced in Sec. 3.3. For Model A (ticket occurrence prediction), the real ticket data are augmented with synthetically generated hard negative samples. For each observed ticket within a given tuple (car model, production batch, function), additional negative samples are

generated in a local neighborhood of integration steps as described in (3.2). This strategy reduces extreme class imbalance and concentrates learning on decision-relevant regions. In addition, lag features are introduced. For each configuration tuple, the severity value of the previous integration step is added as a feature. If no previous ticket exists, the lag severity is set to zero and an additional binary indicator feature `missing_previous_ticket` is set to 1; otherwise it is set to 0. This encoding allows the model to distinguish between the true absence of prior issues and missing historical information.
For Model B (severity prediction), the dataset consists exclusively of real ticket instances ($T = 1$). No synthetic negative samples are generated. Lag features are constructed analogously to Model A. If no prior ticket exists, the lag severity is set to zero and the missing indicator is set accordingly. This preserves temporal information while respecting the conditional nature of severity modeling.

Temporal Train–Test Splitting. Model performance is evaluated using a temporally ordered expanding-window strategy based on the ticket creation timestamp. The earliest recorded ticket defines the starting point of the observation horizon, which is partitioned into quarterly intervals (13-week windows).
Training begins after the fourth production quarter to ensure sufficient historical data, as early integration phases contain sparse ticket information. In the first evaluation step, the model is trained on quarters Q1-Q4 and evaluated on Q5. In the subsequent step, the training set is expanded to include Q5 (Q1-Q5), and predictions are generated for Q6. This procedure continues sequentially, such that the training dataset grows over time while the test quarter always lies strictly in the future.
An offset gap is introduced between the end of each training window and the start of the corresponding test quarter to prevent temporal information leakage. This expanding-window approach reflects realistic deployment conditions, where models are retrained as new data becomes available and are used to predict future validation outcomes.
When evaluating the new architecture, legacy architecture data are incorporated into the training set (where explicitly stated) while preserving strict temporal ordering. All models are retrained separately for each window to ensure consistent and fair comparison.

Feature Encoding. Before model training, categorical variables are transformed into numerical representations. For the Random Forest baseline, we apply one-hot encoding. In one-hot encoding, a categorical variable with K distinct categories is transformed into K binary indicator variables, where exactly one entry is equal to 1 and all others are 0. Formally, for a categorical feature $x \in \{c_1, \ldots, c_K\}$, the encoded vector $\mathbf{e}(x) \in \{0,1\}^K$ is defined as

$$\mathbf{e}_k(x) = \begin{cases} 1 & \text{if } x = c_k, \\ 0 & \text{otherwise.} \end{cases}$$

This representation removes artificial ordinal relationships and allows tree-based models to treat categories independently [16, 48]. Continuous features are left in their numerical form.
For the PFN-based models, categorical features are not one-hot encoded. Instead, they are represented as integer indices and internally projected into a latent embedding space as part of the model architecture. Each feature value is mapped to a learned embedding vector, and labels are encoded as additional tokens, so that the entire dataset (training context and query samples) is processed jointly by a Transformer [20, 23]. This design enables permutation-invariant in-context conditioning without requiring explicit manual feature expansion.

Model Training Procedure. All models are trained separately for each rolling time window described above. The training procedure follows the probabilistic decomposition introduced in Sec. 3.3, where ticket occurrence (Model A) and conditional severity (Model B) are estimated independently.
Prior Classifier. The prior baseline is implemented using the scikit-learn framework and relies solely on empirical class frequencies observed in the respective training split.
Random Forest. Random Forest models are implemented using `RandomForestClassifier` from scikit-learn. Categorical features are transformed using one-hot encoding, which preserves their nominal structure and avoids introducing artificial ordinal relationships. Since tree-based models operate on numerical inputs and are robust to sparse binary features, one-hot encoding provides a stable representation. The option `handle_unknown="ignore"` ensures robustness to unseen categories in future rolling windows.
The classifier is configured with `n_estimators=500`, `max_depth=20`, `class_weight="balanced"`, and `n_jobs=-1`. A large number of trees reduces variance and stabilizes probability estimates, while the depth constraint controls model complexity and mitigates overfitting. Balanced class weighting compensates for class imbalance, and parallelization improves computational efficiency.
To ensure sufficient training signal, the Random Forest is trained on the complete legacy dataset together with the rolling training window of the new architecture. Because Random Forest learns static decision boundaries and does not perform in-context conditioning, training solely on early quarters of the new architecture would result in unstable partitions and limited generalization. Including legacy data provides broader structural coverage while preserving temporal ordering.
TabPFN (Vanilla In-Context Learning).
The TabPFN classifier (Version V2) is instantiated via `create_default_for_version` with `ignore_pretraining_limits=True`. In contrast to Random Forest, no explicit one-hot encoding is applied. TabPFN internally embeds categorical and numerical features using learned feature tokens and column-aware attention mechanisms, enabling permutation-invariant handling of heterogeneous schemas.
In the vanilla configuration, TabPFN operates purely in in-context learning (ICL)

mode. For each rolling window, only the labeled training split of the new architecture is provided as context. No gradient-based optimization is performed for the downstream task. Predictions are generated via a forward pass conditioned on the context set, with default ensemble averaging retained to stabilize probability estimates.

TabPFN (Fine-Tuned Variant).
For the fine-tuned configuration, the pretrained TabPFN is first adapted on the complete legacy dataset using `FinetunedTabPFNClassifier` in batched mode. Fine-tuning minimizes cross-entropy loss on query samples while conditioning on context samples, with early stopping based on validation performance and checkpoint selection of the best model.
After adaptation, inference on the new architecture again follows the in-context learning paradigm. Only the rolling training window of the new architecture is provided as context. Thus, fine-tuning adjusts the implicit task prior toward the automotive validation domain, while downstream predictions remain context-conditioned rather than retrained from scratch.

Hybrid Model. The hybrid configuration combines the strengths of both modeling paradigms. Ticket occurrence probabilities are estimated using the Random Forest trained on legacy and new architecture data, while conditional severity probabilities are obtained from the vanilla TabPFN model. The final expected risk score is computed according to (3.18). This specialization allows nonlinear partitioning for occurrence detection while leveraging probabilistic conditioning for ordinal severity estimation.

Risk Evaluation Protocol. After training, Model A (trained on real and synthetic data) and Model B (trained on real tickets only) are evaluated jointly for risk-based prioritization. For ranking evaluation, predictions of both models are generated on the Model A test dataset. This ensures that severity predictions are also produced for synthetically generated configurations. The final expected risk score is computed according to (3.18). and used to rank configurations.

Probability Alignment and Distribution Correction. In certain rolling windows, not all severity classes $S \in \{1, 2, 3\}$ are observed in the training dataset. Consequently, the trained severity classifier may only learn a subset of classes, resulting in predicted probability vectors that do not span the full domain of severity levels.
To ensure consistency of the risk formulation in (3.18)., predicted probabilities are explicitly aligned to the fixed domain class set $\mathcal{S} = \{1, 2, 3\}$. Let $\hat{\mathbf{p}}(c)$ denote the class probability vector returned by the trained model over the subset of observed classes. We construct a full probability vector $\tilde{\mathbf{p}}(c) \in \mathbb{R}^3$ such that

$$\tilde{p}_k(c) = \begin{cases} \hat{p}_k(c), & \text{if class } k \text{ was observed during training,} \\ \varepsilon, & \text{otherwise,} \end{cases}$$

where $\varepsilon > 0$ is a small constant (e.g. 10^{-12}).

After insertion of missing classes, the probability vector is renormalized,

$$\tilde{p}_k(c) \leftarrow \frac{\tilde{p}_k(c)}{\sum_{j=1}^{3} \tilde{p}_j(c)},$$

ensuring that $\sum_{k=1}^{3} \tilde{p}_k(c) = 1$.

This procedure guarantees that (i) all configurations receive a probability mass over the complete severity domain, (ii) the expected-risk computation remains well-defined, and (iii) numerical instability in log-loss evaluation due to zero probabilities is avoided. The correction does not introduce artificial signal but merely preserves domain consistency under temporary class sparsity.

4.2 Ticket Occurrence Modeling

We evaluate the results of Model A for ticket occurrence prediction using the metrics introduced in Sec. 3.5, namely recall, accuracy, and log loss. For each metric, we plot the quarterly performance for the three car types (A, B, and C), allowing a direct visual comparison of model behavior across time and vehicle categories. In the legend, we additionally report the metric value in the last observed quarter together with the difference (Δ) relative to the prior classifier. Since all models show relatively stable or slightly improving behavior toward the final quarters, the last-quarter comparison provides a compact summary of recent performance.

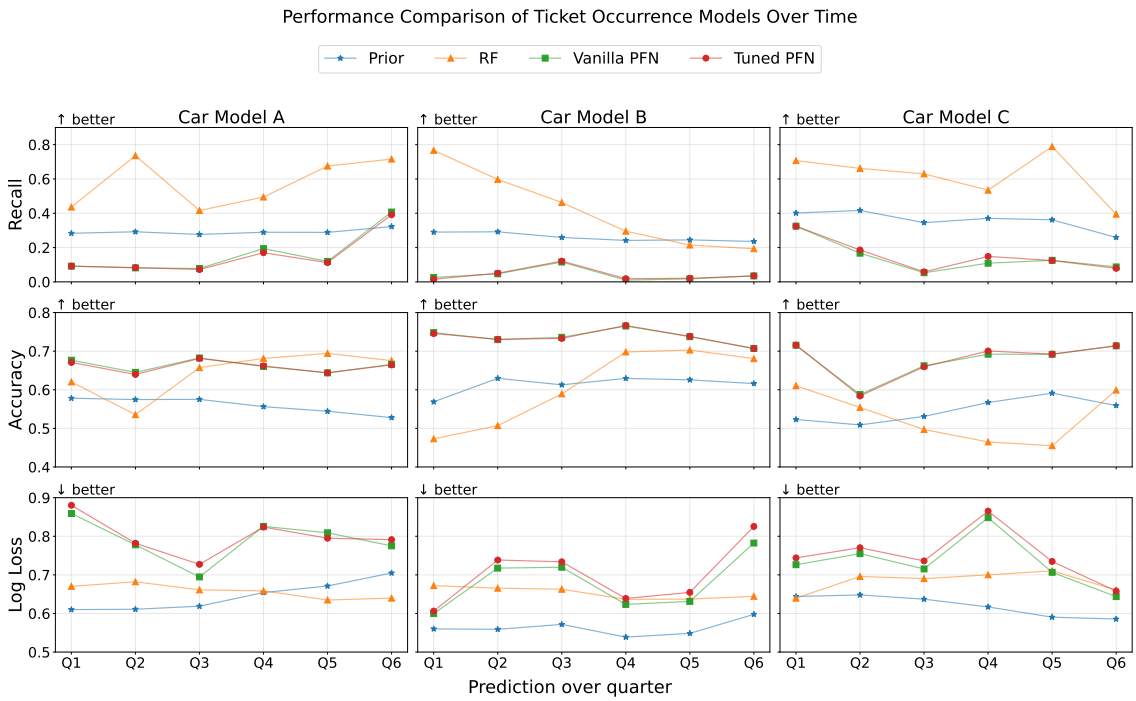

Figure 4.1: Quarterly performance of Model A (Ticket Occurrence) across car types A, B, and C. The figure reports Recall, Accuracy, and Log Loss for Prior, Random Forest, PFN Vanilla, and PFN Fine-Tuned models. The legend indicates the final-quarter performance and the difference relative to the prior classifier.

Recall. Recall measures the proportion of true ticket occurrences that are correctly identified. Across all three car types, the Random Forest achieves the highest recall in the final quarter.
In the last quarter (Q6), Random Forest reaches a recall of 0.716 for Car Model A, 0.193 for Car Model B, and 0.395 for Car Model C. For Car Model C in particular, Random Forest improves substantially over the prior classifier (0.395 vs. 0.259, +0.136 absolute), whereas both the vanilla PFN (0.087) and fine-tuned PFN (0.079) remain clearly below the prior baseline.
A similar pattern is observed for Car Models A and B. For Model A in Q6, Random Forest (0.716) exceeds both the prior (0.322) and the PFN variants (0.407 vanilla, 0.391 tuned). For Model B, Random Forest recall remains consistently competitive, although the margin over the prior becomes smaller in later quarters (e.g., Q6: 0.193 vs. 0.236).
Overall, the Random Forest demonstrates the strongest ability to capture true ticket occurrences, indicating superior sensitivity in detecting ticket events across vehicle types.

Accuracy. Accuracy reflects the proportion of all correctly classified instances, including both ticket and non-ticket cases.

For Car Model A, both PFN variants achieve competitive accuracy in the last quarter, with values of 0.666 (vanilla) and 0.664 (fine-tuned) compared to the prior classifier at 0.528. However, Random Forest reaches the highest accuracy in Q6 with 0.676, indicating only a small gap between the approaches.

For Car Models B and C, the differences between PFN and Random Forest are smaller, and in some quarters the curves nearly overlap. For example, in Model B (Q6), PFN accuracy (0.707) is close to Random Forest (0.681), while for Model C (Q6) PFN (0.714) slightly exceeds Random Forest (0.599).

This suggests that, while PFN models can yield strong overall classification performance, their advantage in accuracy is not uniformly consistent across all vehicle types.

It is also important to note that accuracy is influenced by class imbalance. If non-ticket cases dominate, high accuracy does not necessarily imply strong ticket detection. Therefore, the recall results provide a more informative picture regarding the detection capability for the positive class.

Log Loss. Log loss evaluates the quality of the predicted probabilities rather than only hard classifications.

From the plots, the differences in log loss between models are relatively modest in several quarters, but can exceed 0.01 in individual cases. In the final quarter (Q6), for example, Random Forest achieves a log loss of 0.640 for Model A, compared to 0.705 for the prior and 0.775/0.791 for the PFN variants.

In several cases, both PFN variants and Random Forest perform slightly worse than the prior classifier, although the magnitude of this difference varies by model and quarter. For instance, in Model C (Q6), the prior reaches 0.586 compared to 0.660 (Random Forest), 0.644 (vanilla PFN), and 0.658 (tuned PFN).

This indicates that, despite improvements in recall or accuracy, the probabilistic calibration of the learned models is not consistently superior to the prior baseline.

In particular, the absence of a clear and consistent reduction in log loss suggests that the learned models may improve classification performance without systematically producing better-calibrated probability estimates.

4.3 Severity Modeling

We evaluate Model B, which classifies ticket severity into three ordered classes ($S \in \{1, 2, 3\}$), using Accuracy, Log Loss, and Mean Absolute Error (MAE), as introduced in Sec. 3.5. The quarterly results for all three car models (A, B, and C) are visualized in Fig. 4.2.

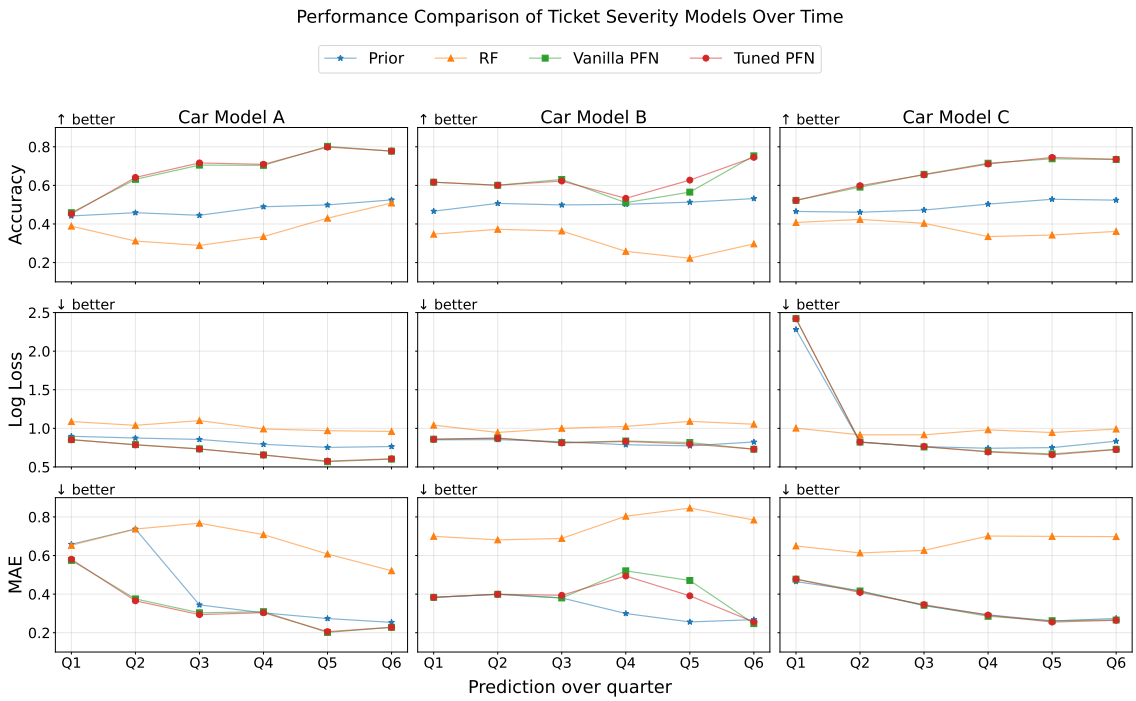

Figure 4.2: Quarterly performance of Model B (Severity Classification) for car models A, B, and C. The figure reports Accuracy, Log Loss, and MAE for Prior, Random Forest, PFN Vanilla, and PFN Fine-Tuned models. The legend shows the final-quarter value and the difference relative to the prior classifier.

Accuracy. Across all three vehicle types, both PFN variants (vanilla and fine-tuned) consistently outperform Random Forest in terms of accuracy.
In the final quarter (Q6), for Car Model A the PFN variants achieve 0.777 (vanilla) and 0.778 (tuned), compared to 0.509 for Random Forest and 0.525 for the prior classifier. A similarly pronounced gap is observed for Car Model B (Q6: 0.753 / 0.746 for PFN vs. 0.297 for Random Forest) and for Car Model C (Q6: 0.734 / 0.735 for PFN vs. 0.361 for Random Forest).
Across quarters, the vanilla and fine-tuned PFN curves remain very close. For example, in Model A (Q6) the difference is only 0.001, and in Model B (Q6) only 0.007. This indicates that fine-tuning provides only marginal additional benefit in terms of classification accuracy.

Log Loss. Log loss evaluates the quality of the predicted probability distributions. For most quarters and car types, PFN models achieve lower log loss values than Random Forest, indicating better probabilistic predictions.
For example, in Car Model A (Q6), Random Forest reaches 0.960, whereas vanilla and tuned PFN achieve 0.602 and 0.605, respectively. A similar pattern appears in Model B (Q6), where PFN achieves 0.729/0.731 compared to 1.054 for Random Forest.

However, for Car Model C in the first quarter, both PFN and the prior classifier exhibit substantially higher loss (prior: 2.280; PFN: 2.422/2.420) compared to Random Forest (1.003). This behavior may indicate sensitivity of the PFN models to distributional differences in early-quarter data.

From the second quarter onward, PFN stabilizes and achieves consistently lower log loss values than Random Forest across all vehicle types.

Mean Absolute Error (MAE). Since severity is ordinal, MAE provides insight into the magnitude of misclassification. Across all vehicle types, PFN models achieve substantially lower MAE than Random Forest.

In the final quarter (Q6), for Car Model B, Random Forest reaches 0.784, while vanilla and tuned PFN achieve 0.247 and 0.255, respectively. For Model C (Q6), Random Forest obtains 0.698, compared to 0.266/0.265 for PFN. Model A shows the same pattern (Q6: RF 0.521 vs. PFN 0.228/0.228).

These results indicate that Random Forest not only misclassifies more often, but also tends to predict severity levels that are further away from the true class. In contrast, both PFN variants maintain consistently low MAE values, suggesting more precise ordinal predictions. Again, the difference between vanilla and fine-tuned PFN remains marginal.

4.4 Risk-Based Prioritization

Since the vanilla PFN and the fine-tuned PFN achieved nearly identical performance across all severity classification metrics, we restrict the risk-based analysis to the vanilla PFN model. Including the tuned variant would not provide additional insight, as its ranking behavior is empirically almost indistinguishable.

The risk-based evaluation consists of three complementary perspectives: (i) the distribution of predicted risk values, (ii) the severity-weighted cumulative gain (3.42), and (iii) Recall@K for high-severity tickets (3.43). We first analyze the distribution of model outputs to understand how the different approaches separate severity levels, and subsequently assess their ranking quality.

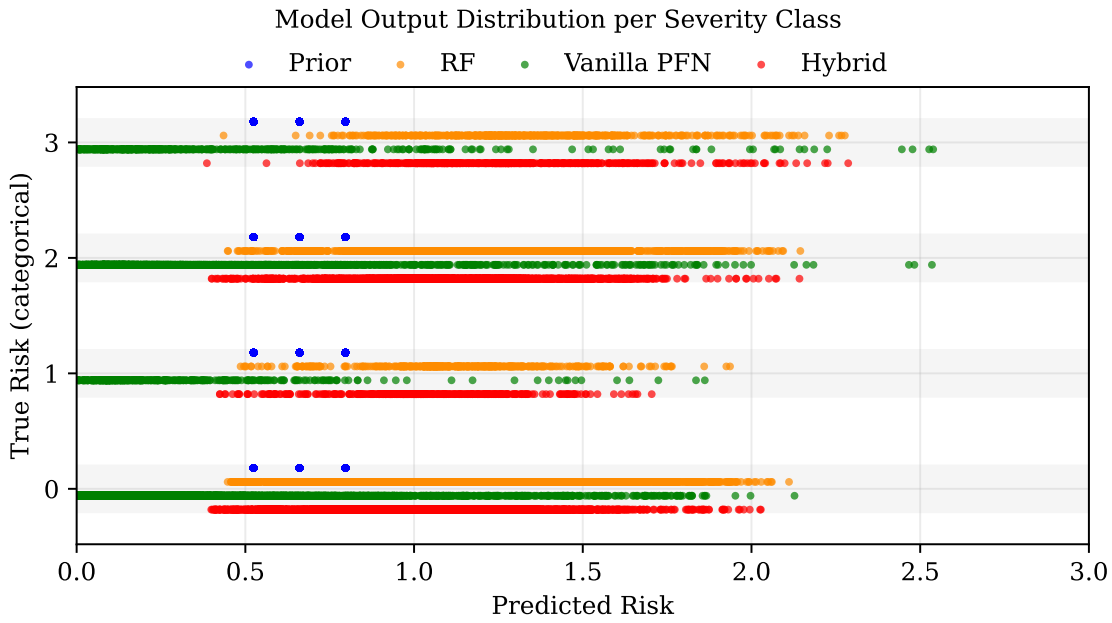

Figure 4.3: Distribution of predicted expected risk scores per true severity class. Each horizontal band represents a true severity level, and the horizontal axis shows the predicted risk value.

Model Output Distribution. Figure 4.3 visualizes the predicted risk scores against the true categorical severity labels. Each horizontal band corresponds to a true severity class, while the horizontal axis represents the predicted expected risk.

A noticeable difference between models is the scale and spread of the predictions. The PFN assigns risk values that extend up to approximately 2.5, while the other models (Prior and Random Forest) operate within a narrower range. However, for ranking-based prioritization, the absolute magnitude of the predicted risk is not decisive. What matters is the relative ordering: high-severity tickets must receive higher scores than low-severity ones.

From the scatter plot, PFN shows clearer vertical separation between severity levels. Higher true severities tend to correspond to higher predicted risks, indicating better monotonic alignment. In contrast, the Prior baseline exhibits substantial overlap between different severity classes, and Random Forest shows weaker separation, particularly between medium and high severity. This qualitative inspection already suggests that PFN should yield superior ranking performance.

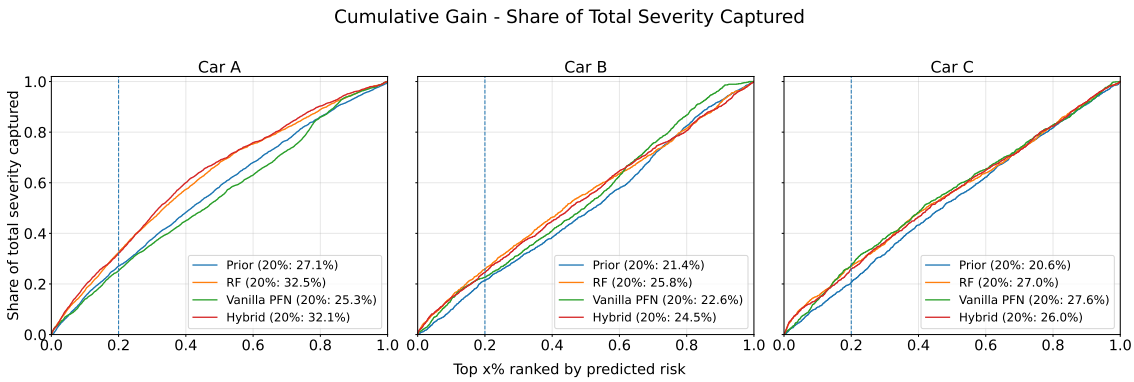

Figure 4.4: Cumulative Gain - Share of Total Severity Captured. The vertical dashed line indicates the 20% inspection budget. Higher curves indicate better concentration of total realized severity in the top-ranked configurations.

Severity-Weighted Cumulative Gain. Figure 4.4 presents the cumulative gain curves, measuring the share of total realized severity captured within the top α fraction of ranked configurations.

For Car A, the Random Forest achieves the highest cumulative gain at the 20% budget (32.5%), closely followed by the hybrid model (32.1%). The PFN performs slightly below Random Forest with (25.3%) but remains under the prior baseline (27.1%). This indicates that, for Car A, Random Forest concentrates a slightly larger portion of total severity within the top-ranked segment.

For Car B, the performance differences are smaller. At 20%, Random Forest reaches 25.8%, compared to 24.5% for the hybrid model, 22.6% for PFN, and 21.4% for the prior classifier. The curves remain relatively close, suggesting that the ranking quality is broadly comparable across models.

For Car C, PFN achieves the highest cumulative gain at 20% (27.6%), slightly outperforming Random Forest (27.0%), hybrid model (26.0%) and clearly exceeding the prior baseline (20.6%). This indicates that PFN is particularly effective at concentrating severe cases early in the ranking for this vehicle type.

Overall, cumulative gain results show that PFN is competitive and in some cases superior, although Random Forest may perform slightly better for certain vehicle types (notably Car A).

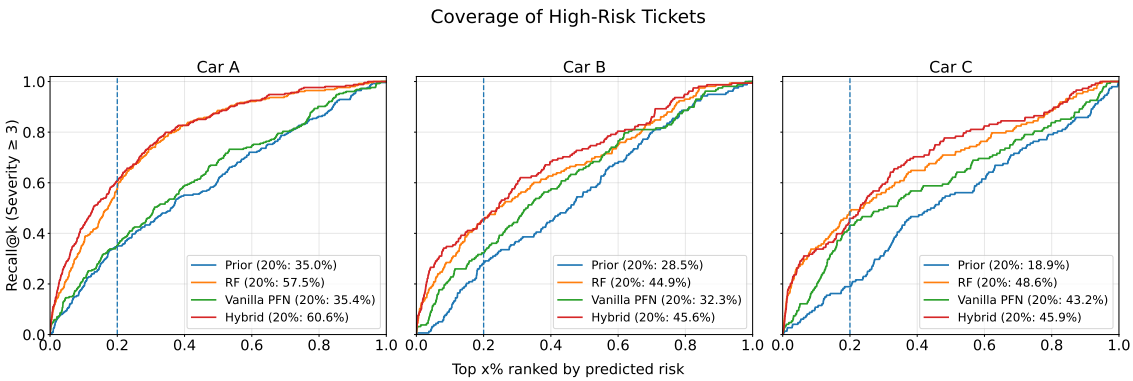

Figure 4.5: Recall@K for high-severity (severity 3) tickets. The vertical dashed line marks the 20% ranking threshold. Higher values indicate better early detection of critical tickets.

Recall@K for High-Severity Tickets. Figure 4.5 evaluates how many of the most critical tickets (severity level 3) are captured within the top-ranked fraction.
For Car A, the hybrid model achieves the highest Recall@20% (60.6%), followed by Random Forest (57.5%). The PFN reaches 35.4%, which is comparable to the prior baseline (35.0%). This indicates that PFN alone is less effective than Random Forest for early detection of the most critical cases in this vehicle type.
For Car B, the hybrid model again performs best (45.6%), slightly outperforming Random Forest (44.9%). PFN achieves 32.3%, remaining clearly above the prior baseline (28.5%) but below the tree-based approaches. The improvement of PFN over the prior remains noticeable but more moderate.
For Car C, PFN and the hybrid model achieve recall levels of 43.2% and 45.9%, respectively. The Random Forest attains 48.6%, while the prior classifier reaches 18.9%. Although PFN and the hybrid model clearly outperform the prior baseline, Random Forest achieves the highest recall for this vehicle type.
Across all vehicle types, the hybrid model consistently outperforms both the prior baseline and the standalone PFN in terms of high-risk ticket coverage. It achieves the highest Recall@20% for Car A and Car B, while Random Forest attains the strongest performance for Car C.

Overall Interpretation The scatter plot indicates that PFN produces better separation between severity levels in terms of predicted risk. Although the absolute risk scale differs across models, ranking quality depends only on relative ordering, not on absolute magnitude. PFN's broader score distribution therefore, does not bias the evaluation, as long as high-severity tickets receive higher scores than lower-severity ones.
Cumulative gain analysis shows that PFN is competitive and, for Car C, slightly superior to Random Forest. For Car A, however, Random Forest achieves marginally higher cumulative gain, indicating that its ranking captures a slightly larger portion

of total severity within the top-ranked segment in that specific setting. Overall, differences in cumulative gain remain moderate across models.
Recall@K provides a clearer distinction. The hybrid model (Random Forest for occurrence prediction combined with PFN for severity estimation) achieves the highest coverage of high-severity tickets, particularly for Car A and Car B. Random Forest generally outperforms the standalone PFN in terms of Recall@K, while PFN contributes improved ordinal and probabilistic severity estimation. The hybrid approach therefore benefits from the strong recall of Random Forest in occurrence prediction and the superior severity modeling capabilities of PFN.
In summary, Random Forest provides stronger early detection performance than PFN alone, whereas PFN clearly outperforms the prior baseline. The hybrid combination yields additional gains in the early identification of critical tickets, indicating that the complementary strengths of both models can be leveraged within a two-stage risk framework. The fine-tuned PFN does not offer a measurable advantage over the vanilla version, and therefore the vanilla PFN remains the preferred standalone model, while the hybrid configuration represents the strongest overall ranking solution.

Chapter 5

Discussion

> "If a man will begin with certainties, he shall end in doubts; but if he will be content to begin with doubts, he shall end in certainties."
>
> — Francis Bacon [49]

5.1 Interpretation of Key Findings

This section interprets the empirical findings from Chapter 4 in light of the research questions and the probabilistic modeling framework introduced in Chapter 3.

Occurrence vs. Severity: Different Modeling Regimes. The experimental results reveal a clear structural difference between ticket occurrence modeling (Model A) and severity modeling (Model B). For ticket occurrence, Random Forest consistently achieves the highest recall across vehicle types, particularly for Car Model C. Since recall directly measures the proportion of true ticket events that are detected, this indicates that tree-based ensembles are highly effective in capturing nonlinear decision boundaries in the occurrence dataset.

This behavior is consistent with established findings in tabular machine learning, where tree-based ensembles such as Random Forests are considered strong baselines for heterogeneous feature spaces [16, 18]. In the present dataset, ticket occurrence appears to be driven by localized nonlinear interactions between configuration attributes and integration stages. Random Forest can partition the feature space adaptively and therefore isolate such interaction effects effectively.

In contrast, PFN models do not consistently outperform Random Forest in recall, although they achieve competitive accuracy. The absence of a systematic log-loss improvement further suggests that, for binary occurrence prediction in this dataset, classical ensemble methods already capture a large portion of the predictive structure.

For severity modeling, the situation is fundamentally different. Across all vehicle types and all three metrics (Accuracy, Log Loss, MAE), PFN clearly outperforms

Random Forest. The advantage is particularly pronounced for MAE, indicating that PFN produces predictions that are not only more accurate but also closer in ordinal distance to the true severity class.
This finding suggests that severity modeling benefits from the probabilistic conditioning mechanism of PFN. Unlike Random Forest, which operates through discrete partitioning, PFN approximates Bayesian posterior predictive inference [22, 23]. This appears advantageous in the conditional multi-class setting, where subtle distributional differences between severity levels must be captured. The improved log-loss performance further indicates better probabilistic calibration in the multi-class regime.

Risk-Based Prioritization and Model Complementarity. The ultimate objective of this thesis is not classification performance per se, but risk-based prioritization. The ranking-based evaluation therefore provides the most relevant assessment. The risk distribution plots demonstrate that PFN produces clearer separation between true severity levels in terms of predicted expected risk. Although absolute risk scales differ across models, ranking quality depends solely on relative ordering. PFN shows stronger monotonic alignment between predicted risk and true severity, which directly translates into improved Recall@K for high-severity tickets.
Cumulative gain results show that Random Forest can be competitive in certain vehicle types (notably Car A), but PFN consistently improves coverage of the most critical tickets at early inspection budgets. This is particularly relevant under constrained validation capacity, where only a fraction of configurations can be tested.
The hybrid model combines the strengths of both approaches: Random Forest for ticket occurrence and PFN for conditional severity. Empirically, this hybrid configuration often achieves the highest Recall@20%, confirming that the two models capture complementary aspects of the data-generating process. Occurrence prediction benefits from strong local partitioning, while severity prediction benefits from probabilistic conditioning and ordinal sensitivity.

Limited Effect of Fine-Tuning. Across both Model A and Model B, fine-tuning the PFN on historical architecture data does not yield systematic improvements over vanilla in-context learning. The performance curves remain nearly identical across quarters.
This observation is consistent with recent studies on tabular foundation models, which report that fine-tuning benefits depend strongly on dataset size and domain similarity [28]. Fine-tuning effectively adapts the implicit task prior encoded in the meta-parameters. However, such adaptation typically requires sufficiently large and distributionally aligned target data.
In the present setting, the available historical architecture data may not provide enough additional signal to meaningfully reshape the learned prior. Furthermore, distributional shifts between architecture generations may limit transfer effectiveness, as discussed in transfer learning literature [42]. Consequently, pure in-context

learning already captures most of the attainable performance.

Process-Level Interpretation. From a process perspective, the findings imply that meaningful prioritization can be achieved using structured tabular metadata alone. No direct software code analysis, no coverage metrics, and no architecture-internal signals were required. Instead, historical ticket records and configuration descriptors were sufficient to learn a risk-oriented ranking.
This indicates that the validation process exhibits statistically learnable regularities across production batches, models, and integration stages. The dynamic modeling framework introduced in Chapter 3 successfully translates these regularities into a probabilistic risk score aligned with the cumulative cost abstraction in (3.1).
Most importantly, the approach produces an explicit, risk-sorted list of candidate tests. This directly operationalizes the objective of minimizing expected realized risk under limited testing capacity, thereby providing a concrete decision-support artifact for validation planning.

5.2 Implications and Comparison with Related Work

Positioning within Test Case Prioritization Research. Classical test case prioritization (TCP) research formalizes prioritization as an ordering problem that maximizes early fault detection [10]. Extensions incorporate fault severity and varying test costs [11], and recent automotive-specific work emphasizes severity-aware prioritization in regression testing [14].
The present work extends this line of research in three ways.
First, instead of relying on handcrafted prioritization heuristics or static severity rules, the approach derives risk scores from probabilistic models trained on historical data. This aligns with the shift toward data-driven validation strategies highlighted in recent automotive CI research [3, 4].
Second, the modeling framework explicitly separates ticket occurrence and conditional severity, leading to a unified expected-risk formulation grounded in quantitative risk theory [39]. This provides a coherent probabilistic foundation for prioritization beyond value-neutral metrics such as APFD.
Third, the work evaluates both classical tree-based ensembles and tabular foundation models within the same industrial pipeline. While Random Forest remains a strong baseline, the PFN-based approach demonstrates that modern tabular foundation models can improve ordinal and probabilistic performance in severity modeling.

Implications for Tabular Foundation Models in Industrial Settings. Recent literature positions tabular foundation models as competitive or superior to classical baselines on small to medium datasets [21, 22]. However, their behavior in highly structured industrial validation data has not been extensively documented.

This thesis provides empirical evidence that:

- PFN can outperform strong classical baselines in conditional multi-class severity modeling.
- PFN improves ranking quality for high-severity cases in risk-based prioritization.
- Fine-tuning does not automatically yield gains when target-domain data is limited or distributionally shifted.

Importantly, Random Forest remains highly competitive for binary occurrence prediction. This suggests that, in the present dataset, the available tabular metadata already contain sufficient signal for classical ensemble methods to learn decision boundaries effectively. The additional column-aware conditioning mechanisms of PFN do not substantially improve occurrence recall in this regime.
Therefore, the results do not indicate that foundation models universally dominate classical methods. Instead, they highlight that different modeling paradigms may be optimal for different subtasks within the same validation pipeline.

Addressing the Identified Research Gap In Sec. 2.3, three research gaps were identified: adaptation in specialized domains, methodological integration of foundation models, and transparency of industrial data processing.
This thesis addresses these gaps as follows:

- **Adaptation Gap:** A dynamic, probabilistic model tailored to cross-classified automotive validation data was developed and evaluated across multiple vehicle types and quarters.
- **Methodological Gap:** TabPFN and Random Forest were compared under identical preprocessing and risk aggregation procedures, providing an industrial benchmark for tabular foundation models.
- **Transparency Gap:** The data preprocessing, hard negative sampling, hierarchical modeling assumptions, and risk aggregation steps were explicitly documented, bridging the gap between raw validation logs and actionable prioritization outputs.

By demonstrating that structured tabular metadata can be transformed into a risk-sorted test list without relying on proprietary code artifacts or opaque heuristics, the work contributes a reproducible and conceptually grounded approach to data-driven automotive validation.
Overall, the findings support the view that risk-based prioritization can be framed as a probabilistic modeling problem on industrial tabular data. Classical ensemble methods remain strong baselines, but tabular foundation models offer clear advantages in ordinal and probabilistic subcomponents, particularly when ranking high-severity events under constrained resources.

Chapter 6

Conclusion

This thesis presented a data-driven approach for risk-based test prioritization in automotive software validation. The validation process was modeled as a probabilistic and dynamic system in which ticket occurrence and conditional severity were estimated separately and combined into a unified expected-risk formulation. This structure enabled a direct transformation of historical validation data into a ranked list of test configurations.

Empirical results show that different modeling paradigms show superior performance depending on the subtask. Random Forest achieved the strongest recall in binary ticket occurrence prediction, while the PFN-based model clearly outperformed classical baselines in multi-class severity prediction, especially with respect to ordinal accuracy and probabilistic quality. In the final ranking task, PFN improved early detection of high-severity tickets, and the hybrid model (Random Forest for occurrence and PFN for severity) achieved the strongest overall performance in terms of critical ticket coverage.

The results demonstrate that meaningful and actionable prioritization can be achieved using structured validation metadata alone. Without relying on source code metrics or coverage information, the proposed framework was able to capture statistically learnable regularities across models, production batches, and integration stages. This confirms that risk-based prioritization can be formulated as a probabilistic modeling problem on industrial tabular data.

However, several limitations must be considered. First, the available industrial dataset contains a high proportion of missing values and documents primarily failed tests, while successful test executions are not explicitly recorded. This introduces potential bias in the construction of negative samples. Second, the analysis is restricted to a specific validation domain and architecture generation. Access to additional data from other test domains, vehicle platforms, or integration steps could improve generalizability. Third, dynamic software updates across integration stages were not explicitly modeled beyond temporal features, although they may introduce structural dependencies between batches.

Future work should therefore investigate explicit modeling of software update events

and integration transitions, for example by incorporating release-level features or state-transition mechanisms. Furthermore, extending the framework to additional validation domains and cross-architecture datasets would allow a more systematic assessment of transferability and robustness. Finally, improved handling of missing data and partial observability in industrial validation databases remains an important direction for strengthening probabilistic risk estimation.

Overall, this work provides a transparent and reproducible framework for transforming historical validation data into risk-aware decision support, contributing to more efficient allocation of limited testing resources in automotive software development.

Appendix A

Validation Workflow

At a high level, development requirements define the expected functional behavior and operational constraints of the software deployed on electronic control units, including both nominal functionality and boundary conditions.
Based on these requirements, the testing team derives reproducible test cases, which are executed either as structured test plans (e.g., test drives) or as automated validation procedures. Each execution produces an observable outcome that is classified as either conforming to the requirement or indicating a deviation.
If a deviation is detected, the behavior is analyzed to determine whether it constitutes a reproducible fault. As illustrated in the validation workflow (Figure A.1), issues are first assessed for validity. Invalid findings are rejected, whereas confirmed faults are documented as structured tickets enriched with contextual metadata such as affected model, function, production batch, timestamp, and estimated severity.
The severity assessment enables prioritization within a centralized database, which represents the primary observable output of the validation process and serves as the interface between testing and development. Tickets are forwarded to the software development team for root-cause analysis and corrective action under practical constraints such as limited capacity and release deadlines.
After fixes are implemented, the updated software is re-integrated and re-tested to verify defect resolution and prevent regressions. If the issue persists, the ticket remains active; otherwise, it is closed.
This iterative test–fix–retest cycle continues throughout system integration. The accumulation and temporal distribution of validated tickets therefore reflect the interaction between requirement complexity, implementation quality, and validation coverage.

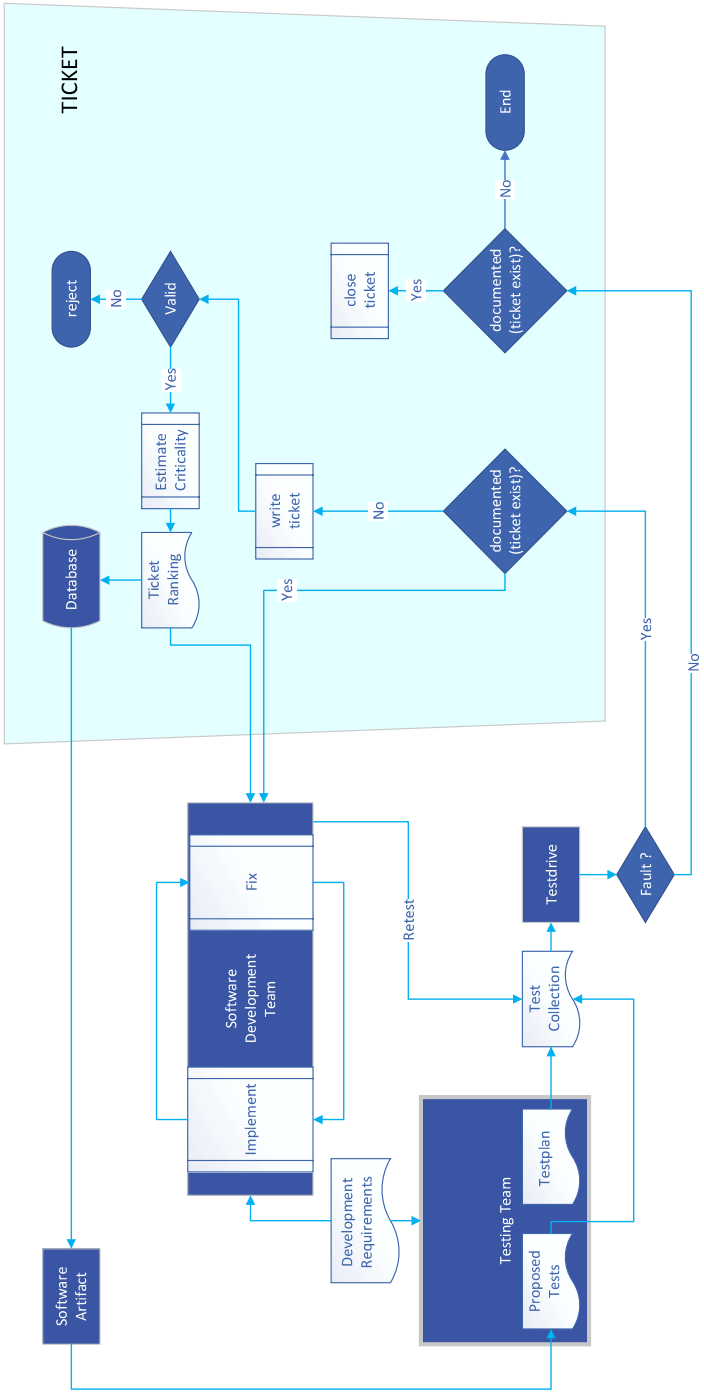

Figure A.1: Comprehensive ticket creation and validation workflow in automotive software development.

List of Figures

3.1 High-level abstraction of the automotive software development and validation process, showing the interaction between requirements, testing, development, integration, and feedback via criticality information. 18
3.2 Three production batches of the same car model, produced in overlapping time intervals with a small temporal offset. Vertical dashed lines mark calendar milestones aligned with the batch endpoints. . . . 20
3.3 High-level architecture of the proposed two-stage data-driven prioritization framework. 34

4.1 Quarterly performance of Model A (Ticket Occurrence) across car types A, B, and C. The figure reports Recall, Accuracy, and Log Loss for Prior, Random Forest, PFN Vanilla, and PFN Fine-Tuned models. The legend indicates the final-quarter performance and the difference relative to the prior classifier. 44
4.2 Quarterly performance of Model B (Severity Classification) for car models A, B, and C. The figure reports Accuracy, Log Loss, and MAE for Prior, Random Forest, PFN Vanilla, and PFN Fine-Tuned models. The legend shows the final-quarter value and the difference relative to the prior classifier. 46
4.3 Distribution of predicted expected risk scores per true severity class. Each horizontal band represents a true severity level, and the horizontal axis shows the predicted risk value. 48
4.4 Cumulative Gain - Share of Total Severity Captured. The vertical dashed line indicates the 20% inspection budget. Higher curves indicate better concentration of total realized severity in the top-ranked configurations. 49
4.5 Recall@K for high-severity (severity 3) tickets. The vertical dashed line marks the 20% ranking threshold. Higher values indicate better early detection of critical tickets. 50

A.1 Comprehensive ticket creation and validation workflow in automotive software development. 60

Bibliography

[1] L. A. Seneca, *Epistulae Morales ad Lucilium.* Ancient Rome, 65. Letter 13.

[2] M. Denninger, A. Richter, M. Held, and B. Rumpe, "Automotive software engineering in an increasingly data-driven automotive sector," *Electronics*, vol. 12, no. 24, p. 518, 2023.

[3] M. Abboush, C. Knieke, and A. Rausch, "Advancing real-time validation of automotive software systems via continuous integration and intelligent failure analysis," *Scientific Reports*, vol. 15, p. 32936, 2025.

[4] Z. Hijab, Zaidi, and U. Ullah, "Machine learning approaches for software defect prediction," *Applied Computational Intelligence and Soft Computing*, vol. 2025, pp. Article ID 7933078, 29 pages, 2025.

[5] L. Rehberg and A. Brem, "Bridging the gap: Linking prototyping and technology readiness levels for integrative product development," *Creativity and Innovation Management*, vol. 34, pp. 237–252, 2025.

[6] Heraclitus, *Fragments.* Ancient Greece, -500. Fragment DK B12 (attributed).

[7] Y. Guo, Y. Chen, H. Wang, and Y. Liu, "Data-driven multivariate time series prediction of in-vehicle equipment failure rates," *Journal of Engineering and Applied Science*, vol. 71, p. 210, 2024.

[8] S. Dillmann and J. R. MartÃnez-Galarza, "Learning representations of event time series with sparse autoencoders for anomaly detection, similarity search, and unsupervised classification," *arXiv preprint*, 2025.

[9] A. Bajaj and O. P. Sangwan, "A systematic literature review of test case prioritization using genetic algorithms," *IEEE Access*, vol. 7, pp. 126355–126375, 2019.

[10] G. Rothermel, R. H. Untch, and M. J. Harrold, "Prioritizing test cases for regression testing," *IEEE Transactions on Software Engineering*, vol. 27, no. 10, pp. 929–948, 2001.

[11] S. Elbaum, A. Malishevsky, and G. Rothermel, "Incorporating varying test costs and fault severities into test case prioritization," in *Proceedings of the 23rd International Conference on Software Engineering*, pp. 329–338, 2001.

[12] Z. Li, M. Harman, and R. M. Hierons, "Search algorithms for regression test case prioritization," *IEEE Transactions on Software Engineering*, vol. 33, no. 4, pp. 225–237, 2007.

[13] F. S. Ahmed, A. Majeed, and T. A. Khan, "Value-based test case prioritization for regression testing using genetic algorithms," *Computers, Materials & Continua*, vol. 74, no. 1, pp. 2211–2238, 2023.

[14] D. Marijan, A. Gotlieb, S. Sen, and M. Tichy, "Severity-aware prioritization of system-level regression tests in automotive software," in *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation (ICST)*, pp. 1–12, IEEE, 2023.

[15] S. H. Zaidi *et al.*, "Advancing real-time validation of automotive software systems using data-driven approaches," *Applied Sciences*, vol. 15, no. 4, p. 2243, 2025.

[16] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning.* Springer, 2 ed., 2009. Chapter 4.4 (Logistic Regression); free PDF available.

[17] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.

[18] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[19] S. Raieli, "Tabular deep learning: A survey from small neural networks to large language models," 2025. TechRxiv preprint.

[20] N. Hollmann *et al.*, "Accurate predictions on small data with a tabular foundation model," *Nature*, 2025.

[21] L. Grinsztajn, K. Flöge, O. Key, F. Birkel, P. Jund, B. Roof, B. Jäger, D. Safaric, S. Alessi, A. Hayler, M. Manium, R. Yu, F. Jablonski, S. B. Hoo, A. Garg, J. Robertson, M. Bühler, V. Moroshan, L. Purucker, C. Cornu, L. C. Wehrhahn, A. Bonetto, B. Schölkopf, S. Gambhir, N. Hollmann, and F. Hutter, "Tabpfn-2.5: Advancing the state of the art in tabular foundation models," 2025.

[22] N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister, and F. Hutter, "Accurate predictions on small data with a tabular foundation model," *Nature*, 2025.

[23] N. Hollmann, S. Müller, K. Eggensperger, and F. Hutter, "Tabpfn: A transformer that solves small tabular classification problems in a second," 2023. ICLR 2023; arXiv version v6.

[24] N. Hollmann, S. Müller, K. Eggensperger, and F. Hutter, "Tabpfn: A transformer that solves small tabular classification problems in a second," 2023. ICLR 2023.

[25] S. Mueller *et al.*, "Transformers can do bayesian inference," 2021.

[26] p. u. a. l. f. a. Zhang *et al.*, "A closer look at tabpfn v2: Understanding its strengths, limitations, and extensions," 2025.

[27] S. B. Hoo, S. Müller, D. Salinas, and F. Hutter, "Extending tabpfn-v2 to time series forecasting," 2026.

[28] I. Rubachev, A. Kotelnikov, and N. Kartashev, "On finetuning tabular foundation models," 2025.

[29] L. Tolstoy, *What Is Art?* New York: Thomas Y. Crowell & Co., 1897. Translated by Aylmer Maude.

[30] B. W. Boehm, "Software engineering economics," *IEEE Transactions on Software Engineering*, vol. SE-10, no. 1, pp. 4–21, 1984.

[31] J. Kim, S. Jin, S. Park, S. Park, and K. Han, "Label-aware hard negative sampling strategies with momentum contrastive learning for implicit hate speech detection," *arXiv preprint arXiv:2406.07886*, 2024.

[32] R. Jiang, T. Nguyen, S. Aeron, and P. Ishwar, "On neural and dimensional collapse in supervised and unsupervised contrastive learning with hard negative sampling," *arXiv preprint arXiv:2311.05139*, 2023.

[33] A. Xu, J.-Y. Hsieh, B. Vundurthy, E. Cohen, H. Choset, and L. Li, "Mathematical justification of hard negative mining via isometric approximation theorem," *arXiv preprint arXiv:2210.11173*, 2022.

[34] A. Gelman, "Multilevel (hierarchical) modeling: What it can and can't do," tech. rep., Columbia University, 2005. Technical report, freely available online.

[35] L. Fahrmeir and G. Tutz, *Multivariate Statistical Modelling Based on Generalized Linear Models.* Springer, 2001. See Chapter 8 (Dynamic Models), in particular Sections 8.1–8.3 on dynamic latent predictors.

[36] L. R. Rabiner, "A tutorial on hidden markov models and selected applications," *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257–286, 1989.

[37] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.

[38] D. Koller and N. Friedman, *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, 2009.

[39] S. Kaplan and B. J. Garrick, "On the quantitative definition of risk," *Risk Analysis*, vol. 1, no. 1, pp. 11–27, 1981.

[40] K. P. Murphy, *Machine Learning: A Probabilistic Perspective*. Cambridge, MA: MIT Press, 2012.

[41] A. Gelman, J. B. Carlin, H. S. Stern, D. B. Dunson, A. Vehtari, and D. B. Rubin, *Bayesian Data Analysis*. Boca Raton, FL: CRC Press, 3 ed., 2013.

[42] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 22, no. 10, pp. 1345–1359, 2010.

[43] M. Sugiyama, T. Suzuki, and T. Kanamori, *Density Ratio Estimation in Machine Learning*. Cambridge University Press, 2012.

[44] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008.

[45] T. Gneiting and A. E. Raftery, "Strictly proper scoring rules, prediction, and estimation," *Journal of the American Statistical Association*, vol. 102, no. 477, pp. 359–378, 2007.

[46] J. S. Cardoso and R. Sousa, "Measuring the performance of ordinal classification," *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 25, no. 8, pp. 1173–1195, 2011.

[47] L. Tolstoy, *Diaries*. Various editions, 1910. Entry on truth and progress.

[48] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.

[49] F. Bacon, *The Advancement of Learning*. London, 1605.